Controller-side feasibility guidance for wind power forecasting under corrupted SCADA

Junyu Li[1], Juntao Du[1,]

[a]*School of Statistics and Applied Mathematics, Anhui University of Finance and Economics, Bengbu, 233030, China*

Abstract

Forecasts used by wind-farm controllers must remain admissible under corrupted SCADA telemetry and must fit batch-1 compute budgets. We study a controller-side feasibility guardrail, denoted FG-MoE, on Wind (CN), where 15-minute SCADA updates support short-horizon reserve, storage, curtailment, and dispatch preparation. FG-MoE turns turbine-envelope information into a training-time regularizer built only from SCADA history available at deployment. An audited sparse carrier (Ultra-LSNT) is used to document timing, memory, and Jump Gate operating-point calibration, while matched iTransformer reruns test whether the same guardrail ordering survives a carrier change. On the primary site, FG-MoE removes above-rated violations that persist in raw outputs without the accuracy collapse caused by clip-only repair. The profiled software loop yields 105.39 ms P95 latency, well inside the 15-minute control cadence. Under one fixed dispatch screen, guarded streams also lower slack burden, positive-error exposure, and real-time cost relative to their matched raw counterparts. These results are specific to Wind (CN) and the reported fixed interface; they are not intended as a claim of universal model or system-wide superiority.

Keywords: Wind power forecasting, Corrupted SCADA, Controller-side feasibility, Operational interface, Feasibility guidance, Robustness

1. Introduction

In typical utility-scale wind-farm operation, controllers receive updated SCADA windows at 15-minute intervals and must issue the next forecast trajectory for reserve preparation, dispatch coordination, storage scheduling, curtailment, or fallback actions. If that trajectory is too

*Corresponding author.

optimistic, reserve can be understated upstream and the shortfall can return later as balancing burden, additional operating cost, or curtailment [**? ? ? ? ?**]. In field operation, this forecasting step is constrained by three engineering conditions: the incoming SCADA stream may be missing, drifting, or corrupted; inference must fit tight batch-1 latency and memory budgets; and the emitted power path must remain admissible before any downstream repair is applied. SCADA quality-control reviews and recent field-scale failure-prediction studies describe these imperfections as routine operating artifacts rather than rare exceptions [**? ? ? ? ? ?**]. The controller-side question is therefore not which model performs best on cleaned benchmarks, but which forecast stream remains usable when corrupted telemetry is all the device receives.

Most reported forecasting gains are still demonstrated after the input series has been cleaned, filtered, reconstructed, or otherwise stabilized [**? ? ? ? ? ? ? ? ?**]. By contrast, SCADA monitoring, diagnostics, and missing-data studies describe an operating environment with missing intervals, duplicates, calibration drift, change points, and operating-region ambiguity, and they usually recover reliability through filtering, anomaly detection, imputation, normalization, or power-curve reconstruction [**? ? ? ? ? ? ? ? ? ? ?**]. This leaves a practical mismatch: benchmarking studies usually compare models after stabilization, whereas controller-side operation starts from the corrupted SCADA stream that actually arrives.

Two further requirements matter once forecasting is moved to the controller. First, edge and industrial-edge studies treat latency, memory, and communication as hard device limits [**? ? ?**]. Second, feasibility-aware and physics-guided forecasting studies suggest that turbine envelopes, power curves, and training-time constraints can move forecasts toward admissible operating regions before post-processing is applied [**? ? ? ?**]. In this setting, sparse computation matters mainly as a controller-budget mechanism rather than as standalone architectural novelty. Because corruption is often localized in time, conditional computation can reallocate processing depth toward unstable windows while retaining a fixed device-side budget [**? ?**].

Operational valuation becomes relevant once the forecast enters the scheduling stack. In short-horizon wind scheduling, an over-rated or under-conservative trajectory can suppress reserve allocation upstream and reappear later as balancing cost or curtailment [**? ?**]. Probabilistic scheduling, market-design, flexibility, and OPF studies further show that the value of the same forecast stream changes with the adopted reserve rule, settlement logic, and forecast-to-grid mapping [**? ? ? ? ? ? ? ? ? ? ? ? ?**]. For the controller-side setting considered here, two gaps are

central:

(1) Forecasting studies rarely evaluate the stream directly on corrupted SCADA under batch-1 controller budgets; most reported gains are still measured after stabilization.

(2) Feasibility guidance and operational valuation are rarely examined together under deployment-available signals and one fixed forecast-to-dispatch interface.

Post hoc clipping or fallback can also blur whether downstream burden comes from the forecast distribution itself or from the repair logic applied after prediction.

To address these gaps, the paper builds a controller-side evaluation chain from corrupted SCADA to one fixed dispatch screen. The methodological focus is FG-MoE, a training-time admissibility guardrail that converts turbine-envelope information into a signal derived only from SCADA history available online. An audited sparse carrier (Ultra-LSNT) is used for timing and memory evidence, while matched iTransformer reruns test whether the same guardrail logic survives a model change. The paper asks a deliberately narrow question: under corrupted SCADA, controller budgets, and one shared downstream screen, does training-time feasibility guidance change the forecast stream that the controller emits? The main contributions are:

(1) We design FG-MoE, a deployable training-time guardrail that enforces turbine-envelope admissibility using only causality-safe SCADA history available at inference time.

(2) We construct a controller-side evidence chain linking corrupted SCADA inputs to one fixed dispatch screen so that burden changes can be traced under a shared downstream interface.

(3) We use an audited sparse carrier plus matched iTransformer reruns to test whether admissibility changes propagate to reserve burden, curtailment, and cost without turning the paper into a carrier-ranking study.

The rest of the paper is organized as follows. Section **??** reviews the adjacent literatures and positions the controller-side gap. Section **??** presents FG-MoE, the audited sparse carrier, and the fixed operational interface. Section **??** summarizes the datasets, corruption protocol, and comparison setup. Section **??** reports the main-site, matched-model, deployment, and dispatch results. Section **??** discusses scope limits, and Section **??** concludes.

2. Related work

The adjacent literatures each address part of the same controller-side problem. The focus here is the intersection of four conditions: corrupted telemetry, feasibility signals available at deployment, controller budgets, and one fixed downstream readout.

2.1. Forecasting on stabilized inputs and SCADA reality

Most wind-forecasting studies begin from stabilized inputs and report substantial improvements after the series has been cleaned, filtered, reconstructed, or otherwise regularized. This includes both decomposition- or representation-heavy forecasting on clean benchmarks and studies that improve long temporal dependence once the input series has already been made analytically stable [**? ? ? ? ? ? ? ? ? ?**].

SCADA studies start from harsher operational assumptions. Missing channels, outliers, calibration drift, regime changes, and operating-region ambiguity are treated as routine, with reliability recovered later through filtering, normalization, anomaly detection, power-curve reconstruction, or imputation [**? ? ? ? ? ? ? ? ? ? ?**]. Fault-log and condition-monitoring studies provide the empirical basis, and preprocessing studies show the usual recovery routes. The missing piece is direct controller-side forecasting on corrupted SCADA before that recovery pipeline has cleaned the signal.

The same contrast also motivates conditional computation, though only as a controller-budget design choice. In live SCADA, drift, flatlines, spikes, and dropouts are often concentrated in specific windows rather than spread uniformly through time. Recent sparse-expert studies in time-series and wind forecasting suggest that expert decomposition can help separate heterogeneous regimes [**? ? ?**], but those studies are usually evaluated on cleaner or server-side settings. Under a fixed controller budget, the issue here is narrower: whether adaptive compute allocation remains practical once telemetry is corrupted and the batch-1 budget is fixed.

2.2. Feasibility-aware and physics-guided forecasting

Power-curve and turbine-envelope studies use cut-in, rated, and cut-out regions to flag implausible SCADA points, reconstruct consistent operating envelopes, and separate turbine behavior from sensor or regime artifacts [**? ? ? ?**]. These constraints sharpen power-curve modeling when raw telemetry contains outliers or mixed operating states [**? ?**]. Performance-analysis

studies using the same envelopes distinguish turbine underperformance from sensor contamination, embedding feasibility information into the operating signal rather than reserving it for post-analysis labeling [**?** **?**].

Physics-guided forecasting extends these ideas into training by incorporating physical priors directly into model learning. Physics-informed reinforcement learning has been used to stabilize probabilistic wind forecasts during extreme events, and trust-oriented studies ask whether forecast explanations and constraints remain reliable enough for deployment [**?** **?**]. Recent reviews point to a shift from post hoc envelope checks toward training-time physical and uncertainty constraints, largely because earlier regularization tends to preserve more accuracy than repairing implausible trajectories after inference [**?** **?**]. The attraction is not only fewer violations, but a forecast distribution that starts closer to admissible operating regions before any repair step is applied.

Many feasibility-aware methods, however, still depend on reconstructed variables, auxiliary states, or richer-than-online signals that may not exist at controller inference time [**?** **?** **?**]. The unresolved issue is how to impose feasibility guidance using signals that remain available online without discarding too much usable forecast range.

2.3. Deployment-consistent forecasting and forecast-to-decision valuation

Edge-computing and industrial-edge studies make the controller environment explicit by treating latency, memory, and communication as first-class design constraints in smart-grid and offshore-wind applications [**?** **?** **?**]. Recent power-system edge reviews sharpen that point by placing smart meters, PMUs, IEDs, and embedded gateways on device layers with limited local computing and storage [**?**]. IIoT collaborative-inference studies and TinyML reviews reach the same conclusion from the embedded side: practical edge AI has to operate under heterogeneous links and tight memory, compute, and power budgets rather than server-class assumptions [**?** **?**]. A forecaster that cannot meet those limits is not a realistic controller option, regardless of benchmark accuracy.

Operational-value studies complete the forecast-to-decision chain by showing how forecast quality translates into system outcomes. One group uses probabilistic forecasts or calibrated uncertainty in reserve procurement and stochastic scheduling [**?** **?** **?**]. Another group evaluates forecast streams through deterministic or scenario-based control, settlement, MPC, OPF, and

coordinated scheduling [**? ? ? ? ? ? ? ? ? ? ? ? ? ?**]. These studies move the literature closer to decision-coupled evaluation by asking how forecast behavior propagates through reserve, control, or network layers rather than stopping at generic error metrics. Together, they show that cost, curtailment, congestion, voltage-security margins, and imbalance exposure depend not only on forecast error statistics but also on the declared forecast-to-grid mapping and the downstream network model. Comparative evidence remains limited for settings that keep corrupted SCADA, deployment-available feasibility signals, controller budgets, and one fixed reserve-to-dispatch chain in the same evaluation setup. Table **??** summarizes the representative strands around that joint setting.

Forecasting papers, SCADA diagnostics, and operational studies each solve part of the problem summarized in Table **??**. Taken together, the table shows one intersection problem: corrupted telemetry, deployable feasibility guidance, controller budgets, and one fixed downstream mapping are rarely kept together in the same study.

3. Methodology

The methodological claim of this paper lies in FG-MoE, a training-time feasibility guardrail built from signals available to the controller at deployment. Corrupted SCADA is first processed by a carrier model, the emitted forecast stream is regularized by the guardrail, and the resulting stream is then evaluated under one fixed dispatch screen. Ultra-LSNT is used here as the audited sparse carrier for timing, memory, and Jump Gate calibration rather than as the paper's transferable contribution; matched iTransformer reruns are used to check whether the same guardrail ordering survives a carrier change.

3.1. Problem setup

Given a multivariate look-back window $\mathbf{X} \in \mathbb{R}^{L\times C}$ with L historical steps and C SCADA covariates, the prediction task aims to forecast an H-step active-power trajectory $\hat{\mathbf{Y}} \in \mathbb{R}^{H\times 1}$. The constrained learning problem is formulated as

$$\min_{\theta} \ \mathbb{E}\Big[\mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}, \mathbf{Y})\Big]. \tag{1}$$

$$\hat{y}_{t+h} \in \mathcal{F}_{\text{phys}}(v^{\text{proxy}}_{t+h}), \qquad v^{\text{proxy}}_{t+h} = g_h(v_{t-K+1:t}), \quad h = 1, \ldots, H. \tag{2}$$

Table 1: Representative literature strands and their remaining gaps relative to the present controller-side setting, which is designed for operation-aware evaluation from corrupted SCADA to a fixed downstream screening readout.

Panel A

Data-side / forecasting-side strands

Study line	Typical assumption	Main strength	Most relevant missing piece
Stabilized-input forecasting after reconstruction or decomposition [**? ? ?**]	cleaned or reconstructed wind series	handles non-stationarity and multiscale variation	does not keep corrupted SCADA as the primary controller-side input condition
Representation-heavy forecasting on clean or server-side benchmarks [**? ? ? ?**]	clean benchmarks or server-class evaluation	improves representation, long dependencies, and transfer	rarely evaluated on corrupted live SCADA under device-side budgets
SCADA diagnostics and preprocessing-dependent recovery [**? ? ? ? ? ?**]	raw SCADA followed by cleaning, segmentation, or filtering	identifies telemetry instability, change points, and abnormal operating regions	usually restores reliability before forecasting instead of keeping a direct corrupted-SCADA forecasting path

Panel B

Deployment-side / decision-side strands

Study line	Operational focus	Main strength	Most relevant missing piece
Deployment-consistent controller studies [**? ? ? ? ?**]	device-side latency, memory, power, and communication limits	makes field-device constraints explicit	rarely combines device budgets with corrupted-input forecasting and one fixed evaluation chain
Feasibility guidance with physical priors [**? ? ? ?**]	envelopes, power curves, and training-time physical discipline	adds feasibility control during or around prediction	often relies on richer-than-deployable signals or does not test the same fixed controller budget
Forecast-to-decision valuation under declared interfaces [**? ? ? ? ? ?**]	reserve procurement, DA/RT settlement, and forecast-to-grid mapping	shows value depends on reserve policy, imbalance settlement, and the declared dispatch interface	rarely keeps corrupted input, controller constraints, and one shared operational interface in the same protocol

Note: The table locates the representative strands discussed in Section **??**. Across them, the recurring gap is the same: data-side input realism, controller-side feasibility with compute discipline, and downstream burden interpretation under one fixed operational interface.

where $\mathcal{F}_{\text{phys}}(\cdot)$ denotes the feasible power envelope and v_{t+h}^{proxy} represents the deployable wind proxy used throughout this investigation. The mapping $g_h(\cdot)$, using only recent observed wind history, remains available online throughout deployment. The envelope, parameterized by turbine specifications (P_{rated}, v_{in}, v_{rated}, v_{out}), is constructed as detailed in Section **??**.

3.2. Carrier front-end for multi-scale SCADA states

Controller-side SCADA mixes slow operating drift with short disturbances such as ramps, flatline release, spikes, and curtailment recovery. The front-end is introduced only to expose these regimes in a computationally cheap form before routing; it is not claimed as a standalone methodological contribution.

The carrier front-end separates slow trend variation from faster residual fluctuations before routing and applies a moving-average operator following decomposition–fusion practice in renewable forecasting [**? ?**]:

$$\mathbf{X} = \mathbf{X}_{\text{trend}} + \mathbf{X}_{\text{season}}, \qquad \mathbf{X}_{\text{trend}} = \text{MA}(\mathbf{X}), \tag{3}$$

where $\text{MA}(\cdot)$ denotes the moving-average operator.

The residual component, embedded through parallel 1D convolutions, is processed as

$$\mathbf{E}_s = \text{Conv1D}_{k_s}(\mathbf{X}_{\text{season}}), \tag{4}$$

$$\mathbf{E}_m = \text{Conv1D}_{k_m}(\mathbf{X}_{\text{season}}), \tag{5}$$

followed by learnable fusion:

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m)\,, \tag{6}$$

where $\phi(\cdot)$ represents a pointwise nonlinearity and $\mathbf{W}_s, \mathbf{W}_m$ are projection matrices. In physical terms, $\mathbf{E}$ acts as a compact state descriptor that retains operating level together with local disturbance cues before expert allocation. This block provides the representation subsequently used for sparse routing.

3.3. Sparse compute allocation under controller budgets

This module serves as the sparse controller-side carrier used in the main experiments. It selects a small subset of experts for each token through Top-K routing, following classical conditional computation with load-balanced gating adapted here to time-series forecasting with a Jump Gate [**? ?**]. In controller terms, the experts are intended to separate approximately steady windows from ramp-dominated, recovery-dominated, or locally irregular windows caused by spikes, dropouts, and regime changes. Top-K routing therefore acts as an online compute allocator: easier windows remain on a small path, while higher-risk windows can recruit additional capacity without forcing the entire stream through the most expensive path.

3.3.1. Routing formulation

Given the decomposition-and-embedding front-end above, the token-wise router input is

$$\mathbf{e}_t = \mathbf{E}[t, :], \tag{7}$$

where $\mathbf{E} \in \mathbb{R}^{L \times d}$ denotes the fused embedding from Section **??** and $\mathbf{e}_t \in \mathbb{R}^d$ represents the representation routed at step t.

Figure **??** provides the single end-to-end workflow view for the main Wind (CN) setting ($L = 96, H = 24$).

Frequency-aware alternatives such as VMD/EEMD can be used in the same interface [**? ?**].

3.3.2. Training objective

Let $\mathbf{e}_t \in \mathbb{R}^d$ denote the embedding at step t. Let $\{E_i(\cdot)\}_{i=1}^N$ denote expert networks:

$$E_i(\mathbf{e}_t) = \sigma(\mathbf{W}_i \mathbf{e}_t + \mathbf{b}_i), \tag{8}$$

where $\mathbf{W}_i \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_i \in \mathbb{R}^d$. Each expert is intentionally lightweight so that routed paths remain inexpensive under batch-1 conditional computation. This is a controller-budget choice rather than a general claim about shallow experts.

A router computes logits and probabilities:

$$\mathbf{z}_t = \mathbf{W}_r \mathbf{e}_t + \boldsymbol{\epsilon}_t, \qquad \mathbf{p}_t = \mathrm{Softmax}(\mathbf{z}_t), \tag{9}$$

where $\mathbf{W}_r \in \mathbb{R}^{N \times d}$ and $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \eta_e^2 \mathbf{I})$. The exploration scale is annealed by epoch index e:

$$\eta_e = \eta_0 \exp(-e/\tau_\eta). \tag{10}$$

Top-K routing selects $\mathcal{S}_t = \mathrm{TopK}(\mathbf{p}_t, K)$ and aggregates expert outputs as

$$\mathbf{o}_t = \sum_{i \in \mathcal{S}_t} p_{t,i} \, E_i(\mathbf{e}_t). \tag{11}$$

Training employs prediction loss and routing regularization.

Point forecasting is optimized by MSE:

$$\mathcal{L}_{\text{pred}} = \frac{1}{H} \sum_{h=1}^{H} \|\hat{y}_{t+h} - y_{t+h}\|_2^2 \,. \tag{12}$$

Appendix **??** reports optional probabilistic heads and routing-based uncertainty diagnostics. The main controller-side training path stays deterministic and does not depend on a rolling residual-calibration layer.

The router load-balancing regularizer is

$$\mathcal{L}_{\text{aux}} = \sum_{i=1}^{N} \left(\bar{p}_i - \frac{1}{N} \right)^2, \qquad \bar{p}_i = \frac{1}{T} \sum_{t=1}^{T} p_{t,i}, \tag{13}$$

and the routing-module objective is

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}}. \tag{14}$$

Up to this point, the carrier only determines how information and compute are organized. The energy-side contribution enters next, where the forecast is penalized against a deployable turbine envelope.

3.4. Feasibility-guided envelope regularization

FG-MoE, the main controller-side guardrail investigated herein, adds a penalty that discourages forecasts outside the turbine envelope, using a SCADA-observable proxy that remains available at deployment.

A horizontal-axis turbine is bounded by

$$P_{\text{aero}}(v) = \frac{1}{2} \rho A v^3 C_p(\lambda_{\text{tsr}}, \beta), \qquad A = \pi R^2, \tag{15}$$

where ρ denotes air density, A represents rotor swept area, v indicates hub-height wind speed, and C_p is the power coefficient with $0 \le C_p \le C_{p,\max} \le 16/27$. The operational envelope, parameterized by $(P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}})$ following standard wind-turbine power-curve conventions and test practice [**? ? ?**], defines the feasible operating region.

Because future wind-speed states are unavailable during deployment, FG-MoE uses a horizon-aware proxy built only from recent observed wind history. We therefore retain the proxy as a symbolic screening template and instantiate it with one site-calibrated parameter set only after identification on the chronological training split. The proxy combines persistence, a short moving average, and a locally clipped trend term:

$$\begin{aligned} v_{t+h}^{\text{proxy}} &= \omega_{p,t} v_t + \omega_{s,t} \bar{v}_t^{(K)} + \omega_{r,t} (v_t + h^{\eta_h} \Delta_t), \\ &\omega_{p,t} + \omega_{s,t} + \omega_{r,t} = 1, \qquad h = 1, \dots, H, \end{aligned} \tag{16}$$

where $\bar{v}_t^{(K)}$ denotes the mean over the recent K observed wind-speed states, Δ_t represents a clipped local slope estimated from the same recent history, and the weights $\omega_{p,t}, \omega_{s,t}, \omega_{r,t}$ adapt to recent volatility and curvature. This construction remains deployable because it does not depend on future or auxiliary wind observations, yet it is less myopic than pure persistence. The structure also follows controller logic rather than full-state reconstruction: the persistence term anchors the proxy to the latest verified operating state, the short moving average damps burst noise and flatline-recovery overshoot that should not be extrapolated point-for-point, and the clipped trend term allows short-horizon ramp anticipation without letting one abnormal increment dominate the whole horizon screen.

The proxy form is fixed, but its cadence- and site-sensitive scalars are calibrated on the chronological 80% training split and then frozen for all retained controller-side comparisons. Under the 15-minute Wind (CN) cadence, $\Delta_{\max}$ caps how far a local ramp can be extrapolated before the proxy becomes implausibly aggressive, s_v sets the volatility level at which weight begins to move from persistence/local averaging toward the ramp term, $(\omega_r^{\min}, \omega_r^{\max})$ prevents the trend branch from either vanishing during genuine ramps or dominating under noisy SCADA bursts, $(\alpha_s, \beta_s, \omega_p^{\min})$ set the smoothing and persistence floors, η_h controls horizon growth, ξ_r mixes the trend and local-mean branches, and s_κ damps high-curvature extrapolation. This proxy is site-calibrated and is not intended as a turbine-invariant physical law. Let $\bar{v}_t^{(K)} = \frac{1}{K}\sum_{j=0}^{K-1} v_{t-j}$ with $K = \min(8, L)$ and let $K_\Delta = \min(4, K-1)$. Using recent increments $\delta_{t-j} = v_{t-j+1} - v_{t-j}$, we define

$$\Delta_t = \mathrm{clip}\left(\frac{1}{K_\Delta} \sum_{j=1}^{K_\Delta} \delta_{t-j}, -\Delta_{\max}, \Delta_{\max} \right), \tag{17}$$

$$\nu_t = \frac{1}{K-1} \sum_{j=1}^{K-1} |\delta_{t-j}|, \qquad \kappa_t = \frac{1}{K-2} \sum_{j=1}^{K-2} |\delta_{t-j+1} - \delta_{t-j}|, \tag{18}$$

where ν_t denotes recent volatility and κ_t indicates recent curvature. The unnormalized weights are

$$\omega'_{r,t} = \mathrm{clip}\left(\frac{\nu_t}{\nu_t + s_v}, \omega_r^{\min}, \omega_r^{\max} \right), \tag{19}$$

$$\omega'_{s,t} = \alpha_s + \beta_s(1 - \omega'_{r,t}), \qquad \omega'_{p,t} = \max\left(\omega_p^{\min}, 1 - \omega'_{r,t} - \omega'_{s,t} \right), \tag{20}$$

followed by normalization

$$\omega_{i,t} = \frac{\omega'_{i,t}}{\omega'_{p,t} + \omega'_{s,t} + \omega'_{r,t}}, \qquad i \in \{p, s, r\}. \tag{21}$$

Given this admissible ramp-weight range, the affine smoothing rule keeps the moving-average branch active while calm windows remain persistence-dominated. We then form a raw trend path

$$\tilde{v}_{t+h}^{\mathrm{raw}} = \omega_{p,t} v_t + \omega_{s,t} \bar{v}_t^{(K)} + \omega_{r,t}\Big(\xi_r(v_t + h^{\eta_h}\Delta_t) + (1-\xi_r)\bar{v}_t^{(K)}\Big), \tag{22}$$

and damp the extrapolation by curvature:

$$d_t = \frac{1}{1+\kappa_t/s_\kappa}, \qquad v_{t+h}^{\mathrm{proxy}} = v_t + d_t(\tilde{v}_{t+h}^{\mathrm{raw}} - v_t). \tag{23}$$

For the reported Wind (CN) case, the frozen training-split calibration is $\eta_h = 0.85$, $\Delta_{\max} = 1.25$ m/s, $s_v = 0.85$ m/s, $\alpha_s = 0.12$, $\beta_s = 0.18$, $\omega_p^{\min} = 0.10$, $\omega_r^{\min} = 0.18$, $\omega_r^{\max} = 0.72$, $\xi_r = 0.70$, and $s_\kappa = 1.25$. These values are fixed after training-split identification and are not tuned on the evaluation horizon. On the training split, $|\delta_t| > \Delta_{\max}$ occurs in only 0.18% of single-step increments, $\nu_t > s_v$ in 0.29% of windows, and $\kappa_t > s_\kappa$ in 0.41% of windows. The chosen thresholds therefore sit in the extreme tail of the site history: ordinary ramps are left untouched, while only spike-scale increments, volatility bursts, and high-curvature oscillations trigger stronger screening. Under this Wind (CN) calibration, the affine smoothing rule keeps $\omega'_{s,t}$ in $[0.170, 0.268]$ and $\omega'_{p,t}$ in $[0.110, 0.552]$, so the moving-average branch never drops out during flatline-recovery segments.

This proxy is deliberately low-order and conservative. It steers forecasts toward an admissible envelope without attempting to recover the full turbine state. Appendix Table **??** perturbs one coefficient group at a time by ±30% while keeping the same seed-42 training/evaluation chain and the same shared high-risk dispatch interface. The burden changes remain modest, which supports the use of the proxy as a site-calibrated guardrail rather than a knife-edge coefficient setting.

For a new site, we retain the same proxy form but recalibrate only this small set of cadence- and site-sensitive scalar coefficients after obtaining $(P_{\mathrm{rated}}, v_{\mathrm{in}}, v_{\mathrm{rated}}, v_{\mathrm{out}})$ from turbine documentation or a short clean SCADA calibration block, then freeze them before controller evaluation. This is a parameter-identification step for the screening surrogate rather than a redesign of the proxy form itself. The Wind (US) boundary reruns motivate this step directly, because the Wind (CN) 15-minute coefficients should not be expected to transfer unchanged to a 5-minute site with different target scaling and turbine characteristics.

Using v_{t+h}^{proxy}, the envelope is

$$P_{\text{env}}(v^{\text{proxy}}) = \begin{cases} 0, & v^{\text{proxy}} < v_{\text{in}}, \\ P_{\text{rated}} \left(\frac{v^{\text{proxy}} - v_{\text{in}}}{v_{\text{rated}} - v_{\text{in}}} \right)^3, & v_{\text{in}} \le v^{\text{proxy}} < v_{\text{rated}}, \\ P_{\text{rated}}, & v_{\text{rated}} \le v^{\text{proxy}} \le v_{\text{out}}, \\ 0, & v^{\text{proxy}} > v_{\text{out}}. \end{cases} \tag{24}$$

Let $\tilde{y}_{t+h}^{\text{raw}} = \hat{y}_{t+h}/P_{\text{rated}}$, $\tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}) = P_{\text{env}}(v_{t+h}^{\text{proxy}})/P_{\text{rated}}$, and let $\mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}}) = 1$ when $v_{t+h}^{\text{proxy}} < v_{\text{in}}$ or $v_{t+h}^{\text{proxy}} > v_{\text{out}}$. FG-MoE regularizes the raw forecast through

$$\begin{aligned} \mathcal{L}_{\text{physics}} = \frac{1}{H} \sum_{h=1}^{H} \Big[& \max(0, -\tilde{y}_{t+h}^{\text{raw}})^2 + \max(0, \tilde{y}_{t+h}^{\text{raw}} - 1)^2 \\ & + \max(0, \tilde{y}_{t+h}^{\text{raw}} - \tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}))^2 \\ & + \mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}})(\tilde{y}_{t+h}^{\text{raw}})^2 \Big]. \end{aligned} \tag{25}$$

For the post hoc clip-only control and the fixed dispatch screening interface, we additionally use the deterministic projection

$$\begin{aligned} \tilde{y}_{t+h}^{\text{clip}} = \text{clip}\Big(& \tilde{y}_{t+h}^{\text{raw}}, \, 0, \\ & \tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}) \Big), \end{aligned} \tag{26}$$

which serves only as the clip-only control and the fixed dispatch screen, not as the main FG-MoE training path. Combining Eqs. (**??**), (**??**), and (**??**), the final objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}^{\text{raw}}, \mathbf{Y}) + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}. \tag{27}$$

For reproducibility, Algorithm **??** summarizes training with imperfection injection and feasibility guidance, Algorithm **??** summarizes edge inference with Jump Gate skipping, and Fig. **??** shows the path from corrupted SCADA measurements to fixed-interface dispatch screening outcomes.

The Stage-4 dispatch block in Fig. **??** is external to FG-MoE training. Each forecast stream is later passed through the same forecast-to-grid mapping and train-split bound screen before burden evaluation, so downstream differences can be attributed to the forecast stream rather than to per-model dispatch retuning. The exact mapping and screening equations are given in Section **??**.

Algorithm 1 Ultra-LSNT training workflow with SCADA imperfection injection.

Input: training samples $(\mathbf{X}, \mathbf{Y})$ with $\mathbf{X} \in \mathbb{R}^{L \times C}$, $\mathbf{Y} \in \mathbb{R}^{H \times 1}$

Hyperparameters: experts N, Top-K, router regularizer λ_r, physics weight λ_2, noise config σ_{cfg}, industrial scale α, Jump Gate threshold τ

$\sigma_{\text{eff}} \leftarrow \alpha \cdot \sigma_{\text{cfg}}$

$\tilde{\mathbf{X}} \leftarrow \text{InjectImperfections}(\mathbf{X}; \sigma_{\text{cfg}}, \sigma_{\text{eff}})$

(Gaussian noise, drift, quantization; see Sec. **??** *for definitions.)*

$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\tilde{\mathbf{X}})$

$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$

for $b = 1$ **to** N_{blk} **do**

  $\mathbf{g} \leftarrow \text{Router}_b(\mathbf{Z})$ *(routing probabilities)*

  $\mathcal{E} \leftarrow \text{TopK}(\mathbf{g}, K)$ *(selected experts)*

  $\mathbf{Z} \leftarrow \text{AggregateExperts}_b(\mathbf{Z}, \mathcal{E})$

  $\mathbf{Z} \leftarrow \mathbf{Z} + \text{TrendInteract}_b(\mathbf{Z}, \mathbf{X}_{\text{trend}})$

end for

$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$

$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{BuildHorizonAwareProxy}(\tilde{\mathbf{X}})$

$\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{\mathbf{Y}}^{\text{raw}}, \mathbf{Y})$

$\mathcal{L}_{\text{aux}} \leftarrow \text{LoadBalance}(\mathbf{g})$

$\mathcal{L}_{\text{physics}} \leftarrow \text{PhysicsPenalty}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$

Update parameters to minimize $\mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}$

Missing figure:

`paper_figures_canonical/main_text/fig2_dispatch_pipeline.pdf`

Figure 1: Workflow from corrupted SCADA windows to fixed-interface dispatch screening. The shared affine mapping and deterministic clipping are used only in the secondary screening interface, not in the FG-MoE training path or the primary controller-side forecast output.

3.5. Conditional skipping for edge inference

The Jump Gate defines token-wise conditional skipping inside the audited sparse carrier. The gate score is

$$I_t = \text{Sigmoid}(\mathbf{w}_g^\top \mathbf{e}_t), \tag{28}$$

Algorithm 2 Edge inference workflow with Jump Gate conditional skipping.

Input: window $\mathbf{X} \in \mathbb{R}^{L \times C}$
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\mathbf{X})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
  if $\text{JumpGate}_b(\mathbf{Z}; \tau) = \text{SKIP}$ **then**
    continue
  end if
  Route experts and aggregate as in Algorithm **??**
end for
$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{BuildHorizonAwareProxy}(\mathbf{X})$
$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
Optional screening: $\hat{\mathbf{Y}}^{\text{clip}} \leftarrow \text{PhysicsClip}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$
Output: $\hat{\mathbf{Y}}^{\text{raw}}$ *(or* $\hat{\mathbf{Y}}^{\text{clip}}$ *for the clip-only control / dispatch interface)*

with hard-threshold transition

$$\mathbf{o}_t = \begin{cases} \mathbf{o}_{t-1}, & I_t < \tau, \\ \text{MoE}(\mathbf{e}_t), & \text{otherwise}, \end{cases} \tag{29}$$

where τ denotes the threshold parameter. In the reported setting, τ is selected from the clean Wind (CN) controller sweep in Appendix Fig. **??**. The edge-frugal setting $\tau = 0.3$ increases skipping but produces a visible clean-slice skill drop, whereas $\tau \geq 0.5$ preserves the same clean-accuracy regime. In deployment terms, τ controls the trade-off between skip rate and software-loop margin: a smaller threshold saves more compute but risks bypassing difficult tokens, while a larger threshold preserves accuracy at the cost of wasting conditional-compute headroom. We therefore fix the smallest threshold that meets the measured latency target without incurring that accuracy loss, $\tau = 0.5$, for all corruption, dispatch, and transfer experiments; it is not learned online at deployment. In other words, τ should be read here as a frozen software operating point for a standardized comparison, not as a load-adaptive PLC/RTU policy. This keeps the compute budget identical across the reported comparisons. A practical next step is to promote τ from a frozen operating point to a controller-margin-aware knob driven by measured device load, queue occupancy, scan-cycle pressure, or control urgency, but that adaptive policy is intentionally left

outside the standardized comparison.

During training, the hard gate is relaxed as

$$\tilde{z}_t = \sigma((I_t - \tau)/s), \qquad \mathbf{o}_t = \tilde{z}_t \cdot \mathrm{MoE}(\mathbf{e}_t) + (1 - \tilde{z}_t) \cdot \mathbf{o}_{t-1},$$

where $s > 0$ is temperature.

Ultra-LSNT-Lite, defined by reducing sparse-module capacity with Top-1 routing, is configured as

$$(N, K, \#\text{MoE blocks}, d) \;\rightarrow\; (2, 1, 1, 96), \tag{30}$$

where d denotes the hidden dimension.

Complexity follows directly. Let d denote hidden dimension. A dense N-expert computation scales as $O(N \cdot L \cdot d^2)$, whereas Top-K routing scales as $O(K \cdot L \cdot d^2)$ with $K \ll N$. If Jump Gate is enabled, the effective token count is multiplied by the non-skip ratio.

3.6. Evaluation metrics and robustness criteria

Primary point-forecasting metrics are R^2, RMSE, and MAE, computed after inverse scaling in the original physical units of each dataset and aggregated over the full horizon and test split. We retain MAPE for completeness, but wind-power series contain frequent near-zero periods, so ratio-based errors can be disproportionately inflated even with numerical safeguards; MAPE is used only as a supplementary diagnostic rather than a headline metric.

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{T}(y_t - \bar{y})^2}. \tag{31}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}. \tag{32}$$

$$\mathrm{MAE} = \frac{1}{T}\sum_{t=1}^{T}|y_t - \hat{y}_t|. \tag{33}$$

$$\mathrm{MAPE} = \frac{100}{T}\sum_{t=1}^{T}\left|\frac{y_t - \hat{y}_t}{y_t + \epsilon}\right|. \tag{34}$$

Here $\bar{y}$ denotes the sample mean of y_t, and ϵ is a small constant used to avoid numerical instability when y_t approaches zero.

Average point errors do not fully characterize controller-side usefulness. We therefore evaluate tail-error, ramp, smoothness, and routing-stability indicators alongside the primary metrics;

for probabilistic variants we additionally report PICP, PINAW, and CRPS. Robustness tests combine continuous perturbations (Gaussian noise, drift, quantization) with structured SCADA faults (flatline, spikes, correlated dropouts). Throughout the main protocol, $\sigma_{\text{eff}} = 0.15$ is treated as the empirical upper-band anchor, while $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ defines an explicit stress envelope.

4. Experimental design and comparison setup

The experiments, organized around one shared pipeline for data preparation, benchmark selection, corruption stress, and dispatch-coupled screening, maintain consistency across all evaluation stages.

4.1. Data, preprocessing, and tasks

The primary case study, investigating ultra-short-term wind-power forecasting using SCADA measurements from a utility-scale wind farm in Xinjiang, China, is designed to reflect operational conditions with sensor uncertainty and non-stationary weather. Inputs include wind and environmental variables, and the target is aggregated active power.

The detailed dataset characteristics, which are summarized in Table **??**, specify the 15-minute sampling cadence, chronological train/test splits, and SCADA degradation protocol used throughout the main experiments.

Table 2: Dataset characteristics and SCADA degradation protocol.

Item	Specification
Panel A	
Primary and boundary-setting wind datasets	
Wind (CN) SCADA (primary)	15-min sampling, 121,528 records, train/test chronology 80%/20% (97,222/24,306), train range 2021-04-02 00:00 to 2022-03-12 15:45, test range 2022-03-12 16:00 to 2022-06-30 23:45, 8 input channels, target active power in kW.
Wind (US) boundary dataset	5-min sampling, 105,120 records, full-year range 2012-01-01 00:00 to 2012-12-31 23:55, 10 input channels (time/covariates) with wind-power target in MW. This dataset is used for transfer-boundary diagnostics and is excluded from the main controller-side claim.
Unified wind-task protocol	Wind datasets use chronological splitting, train-fitted z-score normalization, inverse scaling before metric reporting, and common QC (duplicate removal, missing-value handling, NaN/Inf sanity checks). Main forecasting setup uses lookback/horizon $(L, H) = (96, 24)$.
Panel B	
SCADA degradation and stress protocol (Wind CN main benchmark)	

Table 2: Dataset characteristics and SCADA degradation protocol (continued).

Item	Specification
Gaussian noise (inference-time)	Configured severity $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ with industrial scaling $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, $\alpha = 1.5$, giving effective tiers $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$.
Drift bias (random-walk)	Persistent bias injected as a channel-wise random walk with step scale $\sigma_{\text{eff}} \cdot \text{std}(x)$; this emulates calibration drift and low-frequency sensor offset accumulation.
Quantization noise	Deterministic rounding with step size $\Delta = \sigma_{\text{eff}} \cdot \text{std}(x)$, i.e., $x_t^{\text{quant}} = \Delta \cdot \text{round}(x_t/\Delta)$, to emulate low-precision SCADA/edge telemetry.
Structured SCADA faults	Flatline, spike/heavy-tail outlier, and correlated dropout with contiguous windows; severity levels $\{0.2, 0.6\}$, with level 0.6 used as the severe-certification slice in main robustness tables.
Missingness and interpolation	i.i.d. masking rate $p \in \{0.1, 0.3\}$ with paired `ffill`/linear interpolation (3 seeds, shared masks across models); burst/correlated missingness is represented by structured dropout faults.
Empirical severity anchor	Channel-wise diff-proxy calibration on Wind (CN) yields median 0.045, P90 0.091, max 0.121; therefore $\sigma_{\text{eff}} = 0.15$ is treated as empirical upper-band neighborhood, while $\{0.30, 0.45, 0.60\}$ are explicit stress envelopes.
Dispatch coupling protocol	For dispatch, 15-min Wind (CN) is aggregated to 1-hour resolution; 120-day rolling copper-plate and IEEE RTS-24 DC-OPF screening are both reported under fixed affine scale mapping and fixed feasibility clipping rules.

Table **??** gathers dataset scope, stress tiers ($\sigma_{\text{eff}} \in \{0.15, 0.30, 0.45, 0.60\}$), and dispatch aggregation (15-min to hourly) in one place, so the main comparisons do not shift preprocessing or severity settings from model to model. In the primary Wind (CN) case, $(L, H) = (96, 24)$ corresponds to a 24-hour lookback and a 6-hour forecast horizon on 15-minute SCADA sampling. This choice preserves one full diurnal cycle in the input while matching the short-term dispatch and reserve-screening horizon used later in the paper. For second-site boundary checking, Wind (US) is used to test whether zero-shot reuse fails and whether limited site-specific recalibration can recover usable skill. It is not treated as a second positive validation site for the main controller-side claim. Cross-domain tasks such as Air Quality and GEFCom Load remain outside the main evidence chain. All datasets are split chronologically to prevent leakage. Z-score normalization is fitted on the training split only and inverse scaling is applied before reporting physical-unit metrics. The same quality-control pipeline (duplicate removal, missing-value handling, NaN/Inf checks) is applied across all experiments.

4.2. Baselines and comparison logic

The comparison logic has two layers. The first asks how models behave under identical corrupted-input stress slices. The second asks how fault augmentation changes each model when the forecast carrier is held fixed. LightGBM and Persistence provide transparent tabular and naive-history baselines in the main panel, while extended-context references such as Chronos and Moirai are reported in the supplementary tables. Long-tail hybrids and constrained-budget metaheuristics are moved to Appendix **??** so that the main comparison stays centered on the controller-side question.

LightGBM and Persistence. Both use the same chronological split, history/forecast horizon $(L, H) = (96, 24)$, train-fitted z-score normalization, inverse scaling, and test-time corruption schedule as the deep models. LightGBM uses a multi-output `LGBMRegressor` on flattened history with fixed hyperparameters (500 trees, learning rate 0.05, random state 42, and all CPU threads). It is included as a transparent tabular baseline rather than as a separately tuned search exercise. Persistence simply repeats the last observed target across the forecast horizon.

Controller-side usefulness also depends on tail error, ramp behavior, smoothness, and routing stability; for probabilistic variants we additionally report PICP, PINAW, and CRPS. Latency, parameter count, model size, and an analytical energy proxy complete the same picture, because a useful model must remain robust under corrupted SCADA and still fit batch-1 deployment constraints. Exact perturbation definitions are deferred to **??** [**? ?**].

4.3. Dispatch interface and screening setup

The dispatch layer provides one fixed operational readout of the forecast streams. We fix one forecast-to-grid mapping and one dispatch rule so that changes in cost, curtailment, and fallback frequency can be attributed to the forecast stream rather than to downstream retuning. This layer is intentionally coarser than the controller-side forecasting task. Its role is to show how forecast errors translate into burden under one shared interface, not to replace higher-fidelity system studies. The batch-1 latency target is fixed earlier by the 15-minute field cadence, and broader mapping-sensitivity checks are left to the supplementary material.

The operational model is a 24-hour day-ahead problem with 1-hour resolution, one thermal unit, upward reserve procurement, and storage [**? ? ?**]. Forecast impact is summarized through operating cost, wind curtailment, and feasibility-restoration (fallback) frequency. A two-stage

DA/RT DC-OPF supplement on IEEE RTS-24 is included as a coarse active-power consistency check under linearized physics, not as an AC-feasibility test [**?**]. The retained RTS-24 screen is fully specified in the released evaluation artifacts, including the base case, hourly load profile, wind/slack bus assignments, affine wind-mapping bounds, and day-level plus aggregate outputs.

The dispatch grid is hourly. We therefore aggregate Wind (CN) from 15-min to 1-hour resolution by averaging each four consecutive records (Table **??**). Each forecaster then provides a 24-step daily vector on that hourly grid, and we evaluate Ultra-LSNT, iTransformer, TimeMixer, and DLinear over a 120-day rolling window. A day is tagged as `fallback_x0` when the optimizer can close only with conservative fallback initialization under unchanged algebraic constraints.

Raw forecasts are reported once to show that the dispatch problem does not close directly: in the raw-scale stress check, all models remain fallback-only over the full 120-day window. Comparative dispatch metrics are then reported after one fixed global mapping and deterministic clipping rule are applied to every model output. Because the interface is fixed across models rather than tuned per model, Table **??** functions as an interface-controlled burden screen rather than evidence that any raw model output is directly dispatch-ready.

To keep operating bounds consistent across models in rolling runs, we apply train-split bound clipping to each daily forecast vector:

$$\tilde{w}_t = \mathrm{clip}(s\hat{w}_t, 0, w_{\max}), \tag{35}$$

$$s = \min\left(1, \frac{w_{\max}}{\max_t \hat{w}_t + \epsilon}\right), \tag{36}$$

where $w_{\max} = \max(w_{\text{train}})$ is computed on the hourly training split and ϵ is a small constant. Results under this mapping are reported in Table **??**, and both raw/mapped rolling summaries are included in supplementary evidence.

All monetary coefficients are in CNY. Because absolute totals depend on price coefficients and reference year, we report both absolute values and relative reductions against fixed baselines. With $\Delta t = 1\,\mathrm{h}$, summed kW schedules correspond to kWh energy over the horizon. Dispatch metrics are always computed against realized wind output on the same test day for all models.

A rolling one-sided residual-calibration diagnostic was also tested. Under the current fixed operational interface, that layer remained too unstable for comparative use, so the main text does not rely on it for operational ranking.

Note on network constraints. The copper-plate model isolates how forecast errors propagate to

reserve, curtailment, and operating cost under one deliberately simplified interface, while the retained RTS-24 supplement only checks coarse DC consistency. The reported burden changes are therefore lower-bound interface evidence rather than AC-grade system valuation. DC-feasible schedules are not guaranteed to be AC-feasible, and richer network or settlement constraints can change magnitudes and rankings by introducing additional channels, such as congestion-driven redispatch, nodal price separation, or voltage-support requirements, through which optimistic forecast paths incur cost.

All experiments run in one pipeline with fixed preprocessing, chronological splitting, and consistent metric computation. Full environment/training settings are reported in **??**. Source code is released at https://github.com/b1ue13e/Ultra-LSNT (MIT), and Wind (CN) data plus preprocessing/evaluation scripts are released for end-to-end reproduction.

5. Results

The results first test whether FG-MoE changes controller-side behavior on the main Wind (CN) site under corrupted SCADA. We then examine whether that effect survives a matched model change, audit the deployment carrier used for timing and memory checks, and finally trace the guarded stream through one fixed operational readout. Turbine replay, proxy-sensitivity, and operating-point analyses, retained as supporting diagnostics, reinforce the same interpretation.

5.1. Main-site guardrail ordering under corrupted SCADA

The first research question asks whether a forecast stream remains controller-usable when corrupted telemetry is the only input reaching the predictor. In the matched guardrail comparison, FG-MoE preserves clean and severe-slice R^2 at 0.8747 and 0.8518 while reducing rated-power exceedance from 0.1164% and 0.0741% for the raw model to 0.0000% and 0.0076%. Across the 434,664 predicted points in this evaluation, that corresponds to 506 clean and 285 severe rated-power exceedances for the raw model versus 0 and 10 under FG-MoE. On the clean slice alone, the raw above-rated points sum to 105.98 MWh of forecast energy beyond the declared rated limit, whereas both FG-MoE and clip-only reduce that quantity to zero. The difference is that clip-only reaches zero exceedance only after collapsing R^2 to 0.4319 and 0.3783. Because these envelope violations would still require screening or correction before controller use, we treat them as controller-side failures rather than harmless numerical excursions. Within one

deployable chain, training-time feasibility guidance is therefore materially stronger than post hoc clipping under corrupted input.

The local-context rows in Table **??** provide a six-model reference frame without turning the paper into a ranking exercise. At the severe Gaussian slice, iTransformer reaches $R^2 = 0.8717$, Persistence 0.8612, TimeMixer 0.8337, Ultra-LSNT 0.7863, LightGBM 0.7121, and DLinear −0.9903. The main point is that Ultra-LSNT maintains a positive corrupted-input baseline inside the audited sparse carrier, whereas a simpler local baseline collapses. Extended-context references are left to the supplementary tables.

The matched-control rows sharpen the same point. When the model is held fixed and only the training condition changes, fault augmentation improves severe-slice R^2 from 0.2402 to 0.3471 for DLinear, 0.7452 to 0.8417 for TimeMixer, 0.8478 to 0.8580 for iTransformer, and 0.8135 to 0.8180 for Ultra-LSNT. The gains, largest in the weaker baselines, indicate that Ultra-LSNT moves only modestly because its corrupted-input baseline is already more stable. The table therefore says less about model ranking than about how training-time feasibility guidance changes corrupted-input behavior, with effect size depending on the model.

At the empirical anchor $\sigma_{\text{eff}} = 0.15$, the intermediate-slice evidence for the six local context baselines remains in positive-skill territory (DLinear 0.7814, LightGBM 0.9340, Persistence 0.9140, Ultra-LSNT 0.8687, TimeMixer 0.8847, iTransformer 0.8993), matching the calibration reported in **??** (median 0.045, P90 0.091, max 0.121). The main tables, therefore, use $\sigma_{\text{eff}} = 0.15$ as the empirical anchor and $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ as explicit stress levels.

Figure **??** visualizes this degradation trajectory across the full stress envelope. The curve reveals a clear separation: DLinear collapses catastrophically to negative R^2 under severe corruption, while Ultra-LSNT maintains positive skill throughout the envelope, albeit below iTransformer and TimeMixer on absolute terms. This pattern clarifies why Ultra-LSNT is retained here: not as the strongest clean-slice model, but as a deployable sparse carrier that stays stable under controller-side stress.

These context rows also keep the carrier choice in perspective: Ultra-LSNT is not the strongest clean, severe-slice, or mapped-dispatch model in this study. It is retained because the paper needs an audited sparse carrier for controller-side deployment and timing evidence, whereas FG-MoE is the main methodological variable.

Missing figure:

paper_figures_canonical/main_text/fig2_robustness_decay_final.pdf

Figure 2: Robustness decay under Gaussian SCADA corruption. Panel (a) focuses on the deep model comparison, showing that Ultra-LSNT maintains positive skill ($R^2 = 0.786$) at severe corruption ($\sigma_{\text{eff}} = 0.60$), albeit below iTransformer and TimeMixer. Panel (b) provides the full performance envelope, highlighting the catastrophic collapse of DLinear to negative R^2 and the intermediate robustness of LightGBM. Operating zones: nominal ($\sigma_{\text{eff}} \leq 0.15$), degraded ($0.15 < \sigma_{\text{eff}} \leq 0.30$), and severe ($\sigma_{\text{eff}} > 0.30$).

Table 3: Robustness evidence on Wind (CN).

Panel A

Clean-trained local context panel (6 models)

Model	**Clean R^2**	**Clean RMSE**	**R^2@0.60**	**RMSE@0.60**	**Rel. RMSE inc. (%)**
DLinear	0.9494	7406.87	-0.9903	46449.77	527.12
TimeMixer	0.8886	8498.38	0.8337	10382.59	22.17
iTransformer	0.9012	8004.05	0.8717	9117.01	13.90
Ultra-LSNT	0.9346	8419.91	0.7863	15218.84	80.75
LightGBM	0.9563	6880.91	0.7121	17665.50	156.73
Persistence	0.9175	7313.94	0.8612	9485.55	29.69

Panel B

Matched-control severe-slice panel

Model	**Clean-trained R^2@0.60**	**Fault-augmented R^2@0.60**	**Gain**
DLinear	0.2402	0.3471	+0.1069
TimeMixer	0.7452	0.8417	+0.0965
iTransformer	0.8478	0.8580	+0.0102
Ultra-LSNT	0.8135	0.8180	+0.0045

Note: Panel A keeps the six-model local-context view at the clean and severe Gaussian slices. RMSE stays in the main table, whereas MAE is moved to **??**. The realistic operating anchor is $\sigma_{\text{eff}} = 0.15$, and $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ marks the stress envelope. Extended-context reference rows such as Chronos and Moirai are left to the supplementary tables rather than the main comparative panel. Full row coverage is listed in the supplement and repository materials.

5.2. Deployment feasibility of the audited sparse carrier

The deployment evidence is specific to the audited Ultra-LSNT carrier and comes from a dedicated paper mode that exits immediately if either the real Wind (CN) file or the released checkpoint is missing. Table **??** thus reports only artifact-backed results from the March 18, 2026 deployment bundle. Starting from Ultra-LSNT-Full in native FP32 form, INT8 quantization

lowers peak memory from 16.70 to 2.53 MiB, trims P95 latency from 6.69 to 6.27 ms, and keeps mean absolute drift at 0.0085. Carrier simplification and runtime export then push the bundle into the low-latency regime: the reduced-capacity sparse carrier (Ultra-LSNT-Lite), exported through ONNX runtime, achieves 0.67 ms P95 latency with 0.75 MiB peak memory, while mean absolute drift against native FP32 inference remains 0.0233.

Jump Gate is treated as a calibrated operating point inside the same carrier family rather than as a separate comparative claim. Across 12 preset-seed Wind (CN) runs, clean-set skip ratios remain nonzero (0.017–0.203) while clean R^2 stays within 0.924–0.933. In the threshold sweep of the selected run, $\tau = 0.5$ targets a skip regime of 0.12 and yields an average skip rate of 0.1155 with an average compute ratio of 0.20. The selected configuration (`aggressive_seed52`) reaches clean $R^2 = 0.9322$, RMSE 6628.6, and a clean skip rate of 0.1066. The "No-Skip upper bound" row in Table **??** is a compute-accounting reference rather than a preferred deployment point.

At the software-loop level, the primary run yields a total P95 of 105.39 ms, including 0.21 ms preprocessing, 6.22 ms inference, 0.10 ms postprocessing, and 99.18 ms dispatch screening. Relative to a 15-minute rolling cadence (900,000 ms), the remaining deadline margin is 899,894.61 ms. A shorter repeat run in the same paper-mode bundle gives a more conservative total P95 of 231.27 ms; even that remains three orders of magnitude below the scheduling horizon. These numbers indicate software-stack headroom under the reported runtime rather than direct evidence on PLC scan-cycle integration, fieldbus overhead, RTOS scheduling, or board-level contention on target controller hardware. They do not establish broad hardware superiority, but they do show that the reported controller bundle is executable under the stated stack. A compact deployment trade-off view that pairs latency–accuracy with latency–memory is moved to Appendix **??**.

5.3. Operational readout under the fixed interface

The operational layer is intentionally narrower than the robustness layer. Under the shared high-risk interface, the issue is whether training-time guidance changes the burden carried by a matched stream. Table **??** shows that, relative to the matched Ultra-LSNT raw stream, the full-guardrail variant lowers slack burden, positive-error exposure, p95 positive error, and total real-time cost while improving high-risk R^2. Slack burden falls from 791.06 to 567.03 MWh and total real-time cost from 30.88 to 30.44 M CNY, whereas the clip-only variant stays close to the

Table 4: Paper-mode deployment proxy and software-loop evidence from the March 18, 2026 bundle.

Panel A

Batch-1 deployment proxy candidates

Model	Backend / Precision	P95 (ms)	Peak Mem. (MiB)	RMSE	Mean Abs. Drift
Ultra-LSNT-Full	PyTorch FP32	6.69	16.70	0.6124	0.0000
Ultra-LSNT-Full	PyTorch INT8	6.27	2.53	0.6128	0.0085
Ultra-LSNT-Full	No-Skip upper bound	3.43	0.39	0.6124	0.0000
Ultra-LSNT-Lite	PyTorch FP32	1.60	0.07	0.6826	0.0000
Ultra-LSNT-Lite	PyTorch INT8	2.19	0.00	0.6818	0.0071
Ultra-LSNT-Lite	ONNX FP32	0.67	0.75	0.6888	0.0233

Panel B

Software-in-the-loop timing envelope (P95)

Run	Prep / Infer / Post (ms)	Dispatch (ms)	Total (ms)	Margin (ms)
Primary run (200 steps)	0.21 / 6.22 / 0.10	99.18	105.39	899894.61
Repeat run (50 steps)	0.37 / 137.95 / 0.19	105.98	231.27	899768.73

Note: All rows are produced in paper mode from the deployment bundle and fail immediately if the real Wind (CN) data file or released checkpoint is missing. Drift is measured against native PyTorch FP32 outputs on the same inputs. The "No-Skip upper bound" row is kept only as a compute-accounting reference; separate controller-side search runs report nonzero clean-set skip ratios while preserving the same clean-accuracy regime. Panel B evaluates the full software loop against the 15-minute rolling-schedule deadline of 900,000 ms and should be read as software-loop timing headroom rather than board-level PLC/RTU evidence.

raw stream. This points to training-time reshaping of the forecast distribution, not to post hoc truncation alone.

The matched decoupling screen on Wind (CN) shows that this burden-reduction pattern is not exclusive to Ultra-LSNT. When the same training-time guardrail is attached to iTransformer under the same interface, the burden metrics move in the same direction and the high-risk R^2 also improves. Slack burden, for example, decreases from 360.97 to 239.57 MWh. This matched rerun supports a mechanism-level reading of the operational gain and strengthens the interpretation of FG-MoE, rather than Ultra-LSNT alone, as the main contribution at the operation layer.

Within the sparse deployment carrier, these burden reductions do not come from giving up usable skill. Across the three Ultra-LSNT guardrail retraining runs, clean-set R^2 stays in the same regime as the matched raw stream while the rated-power exceedance rate falls from 0.1164% to effectively zero. Under severe Gaussian corruption, the guarded runs also retain the raw ac-

curacy regime while materially reducing over-rated predictions. The repeated-seed summary preserves the same burden ordering, and so does the site-calibrated proxy parameter sensitivity rerun. Across the reported ±30% perturbations, high-risk slack and total real-time cost remain tightly grouped (Appendix Table **??**). This supports the Wind (CN) proxy as a site-calibrated guardrail rather than a knife-edge coefficient choice.

The broader 120-day mapped screen answers a different question. It compares model behavior after the shared mapping-and-clipping interface has already been imposed. Under that screen, iTransformer still leads the mapped ranking, while Ultra-LSNT remains materially stronger than DLinear (Table **??**). The raw-scale closure check is unchanged: before mapping, all four models remain in `fallback_x0` for all 120 days. This screen is retained as a fixed controller-to-dispatch readout rather than as a reversal of the mapped ranking or a substitute for higher-fidelity AC or market-settlement validation. It provides lower-bound evidence under a simplified interface: richer network or settlement models may change magnitudes and rankings while adding further channels, such as congestion-driven redispatch or nodal price separation, through which optimistic forecast paths can incur burden. Within that readout, the evidence still supports the narrower point that training-time feasibility guidance changes burden within a matched forecast stream under one shared interface. That matched-stream ordering also survives a reserve-rule sweep in the same mapped copper-plate screen. Replacing the baseline rule with $\sigma_{w,t} = \rho|\hat{w}_t|$, $\rho \in \{0.1, 0.2, 0.3\}$, leaves Ultra-LSNT + FG-MoE below the matched raw stream by 11.3%–24.0% in total cost, while the clip-only control stays effectively on top of raw and the matched iTransformer rerun keeps the same sign at much smaller scale (Appendix Table **??**).

Figure **??** translates the fixed-interface outcomes into visual form. Panel (a) shows total real-time cost across the four mapped baselines, revealing a 44% cost reduction from DLinear to iTransformer. Panel (b) shows curtailment volumes, where the spread is even more pronounced (71% reduction). Ultra-LSNT occupies the middle ground: materially stronger than DLinear while remaining within the deployable sparse-carrier envelope.

5.4. Routing diagnostics for operational interpretation

The sparse routing module is reported as an operational diagnostic on how computation is redistributed under controller stress, not as a one-to-one identifier of physical regimes. To verify that this conditional computation remains interpretable at that limited level, we examine routing behavior at two scales: diurnal composition across tasks and event-window temporal dynamics.

Table 5: Operational readout under the fixed interface for the 120-day rolling test.

Panel A

Fixed-interface comparability / fallback gate

Model	Raw opt.	Raw FB (%)	Mapped opt.	Mapped FB (%)
DLinear	0.0000	100.0	0.3750	62.5
TimeMixer	0.0000	100.0	0.7833	21.7
iTransformer	0.0000	100.0	0.8333	16.7
Ultra-LSNT	0.0000	100.0	0.7583	24.2

Panel B

L1 mapped-protocol outcomes (copper-plate)

Model	L1 Cost (M CNY)	L1 Curt. (M kWh)	L1 FB (%)
iTransformer	32.16	15.22	16.7
TimeMixer	34.90	19.60	21.7
Ultra-LSNT	35.97	22.09	24.2
DLinear (base)	54.47	55.71	62.5

Note: Panel A reports raw-vs-mapped optimization and fallback rates. Panel B gives the fixed-interface point readout under the same mapping-and-clipping rule. The coarse RTS-24 DC-OPF consistency check is left to the supplement and is not used as a main-text ranking layer.

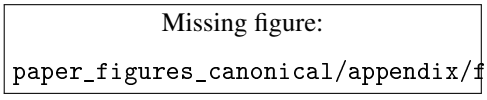
Missing figure:

paper_figures_canonical/main_text/fig3_dispatch_economics_v7.pdf

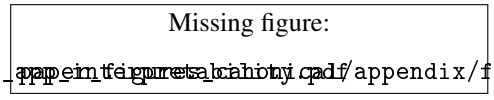
Figure 3: Fixed-interface operational outcomes from the 120-day rolling test under L1 copper-plate dispatch. Panel (a): Total real-time operating cost (M CNY). Panel (b): Wind curtailment volume (M kWh). All values are computed under the shared mapping-and-clipping interface. Ultra-LSNT occupies an intermediate burden position between iTransformer and DLinear while remaining within the deployable sparse-carrier envelope.

At the diurnal scale, activation shares remain bounded without collapse spikes (Fig. **??**, Panel a), supporting the stability of conditional computation. Cross-task mean shares shift in a structured way between Wind (CN) and Air Quality, consistent with task-dependent routing reuse rather than expert collapse.

At event windows, routing redistribution is observable and can be logged as a monitoring signal for anomaly triage. Panel (b) of Fig. **??** shows Top-K routing weights over a representative 64-hour controller window, overlaid on realized and predicted power trajectories. Event markers A and B denote the two highest-ramp windows; the residual inset exposes local lag/error



(a) Diurnal expert shares and cross-task composition.



(b) Event-window routing heatmap over a representative controller slice.

Figure 4: Sparse routing diagnostics for operational interpretation. Panel (a): Diurnal expert activation shares on Wind (CN) and cross-task mean composition, with the 25% uniform-routing reference line indicating balanced allocation. Panel (b): Top-K routing weights over a representative 64-hour controller window, overlaid with realized and predicted power trajectories. Events A and B mark the two highest-ramp windows; the residual inset reveals local lag/error structure. Routing redistributes capacity dynamically without expert collapse, supporting stable conditional computation under controller stress.

structure. In controller terms, the redistribution is most consistent with windows where abrupt ramps, wake-interaction shifts, transient curtailment transitions, or telemetry disturbance make the SCADA segment harder to process with a uniform path. These associations are intentionally hypothesis-generating rather than definitive physical labels.

6. Discussion and scope limits

6.1. What the main evidence chain establishes

Controller-side forecast quality matters before any dispatch model is applied. The forecast stream has to remain usable under corrupted telemetry, meet the compute budget, and stay admissible under the feasibility guardrail. If any one of those conditions breaks, later dispatch differences mostly reflect repair and fallback logic rather than the forecast distribution itself.

Within that controller-to-dispatch chain, sparse routing is a deployment choice rather than a universal model claim. SCADA corruption is heterogeneous in time, with missing segments, change points, and failure-related drifts concentrated rather than uniformly distributed [**? ? ? ?**]. Under a fixed controller budget, a dense always-on pathway assigns identical computation to stable and unstable intervals. The audited sparse carrier instead redistributes computational paths, allocating deeper processing to disturbance-prone windows while retaining lighter pathways for stable intervals. That role matters, but it remains secondary to the main result on controller-side feasibility guidance under corrupted SCADA.

6.2. FG-MoE as the main methodological contribution

FG-MoE differs from clip-only repair because the envelope acts during training rather than only truncating the output after prediction. Recent wind power-curve preprocessing and physics-guided forecasting studies use physical priors to reshape learning before inference [**? ? ?**]; the practical question here is whether that can be done with signals still available online. On Wind (CN), the answer is positive but narrow. FG-MoE stays close to the raw accuracy regime, removes the clean-slice above-rated points before any dispatch mapping, and reappears downstream as lower slack burden and lower positive-error exposure relative to the matched raw stream. The horizon-aware proxy is central to that trade-off. It stays deployable at inference time by combining recent level, local mean, and short-trend cues, but it can still underrepresent forward-looking wind evolution during rapid regime changes. The guardrail is a deployable feasibility prior rather than a full physical simulator.

The matched iTransformer reruns keep this interpretation from collapsing back into a carrier-specific story. On Wind (CN), attaching the same training-time guardrail to iTransformer preserves the same raw/clip-only/full-guardrail ordering at the operation layer. That result does not imply identical gains across models, but it does justify a cleaner contribution statement: FG-MoE is the main methodological contribution, whereas Ultra-LSNT is the audited sparse carrier used to demonstrate controller-side deployment and timing feasibility.

6.3. Operational interpretation under the fixed interface

The operational layer does not ask which model yields the highest overall system value across all downstream settings. Under the fixed interface used here, the narrower issue is whether training-time feasibility guidance changes the burden carried by a matched forecast stream. Once the interface is shared, success and feasibility-fail rates largely saturate, so slack, positive-error exposure, real-time cost, and related burden measures become more informative than binary closure counts. The main operational result is therefore within-stream: FG-MoE lowers burden relative to raw and clip-only under the same interface. It does not follow that the preferred carrier must also become the strongest dispatcher overall.

That restriction matters because AC feasibility, voltage stability, congestion, and settlement rules can magnify or mute the effect of the same forecast error under different downstream models [**? ? ?**]. Holding one interface fixed keeps the screening comparison interpretable; changing

the interface would answer a different question. The reported copper-plate results are therefore a lower-bound screening result under a simplified interface, not a full-network valuation.

Rolling one-sided calibration is left to the supplement because, under the current fixed interface, it did not provide a stable comparative ranking. The main operational interpretation therefore stays on the fixed-interface point screen. Under that screen, iTransformer still leads the broader mapped ranking. The claim kept in the main text is narrower: training-time feasibility guidance changes the burden profile within a matched stream under one shared interface. The RTS-24 supplement is included only as a coarse network-consistency check, not as evidence on AC feasibility, voltage security, congestion management, or market settlement. Once richer network constraints enter, optimistic forecast paths gain extra channels through which they can trigger redispatch, reserve use, curtailment, congestion rents, or nodal price separation; the reported burden reductions should therefore be treated as conservative rather than exhaustive.

6.4. Second-site boundary and remaining limits

Supporting evidence outside the main split first comes from the independent turbine replay. Its observed-fault bank contains 343 recorded events, dominated by 247 wind-speed flatlines and 50 wind-speed spikes, and the 10-seed safety repeat preserves positive FG-MoE clean/severe mean R^2 of 0.7038/0.6801 while raw and clip-only remain negative. These rows extend the controller-side safety ordering beyond the main Wind (CN) split and beyond one training seed, but they do not amount to a second main-site validation.

The matched reruns also sharpen the new-site boundary. On Wind (CN), attaching the same training-time guardrail to iTransformer preserves the raw/clip-only/full-guardrail ordering, indicating that the mechanism is not exclusive to Ultra-LSNT. Wind (US), however, marks the point where zero-shot reuse fails rather than supplying a second positive validation site. In the zero-shot reruns, the transferred guardrail drives cut-region nonzero rates to 57.6–79.8% for Ultra-LSNT and 95.7–100.0% for iTransformer. This failure is consistent with the proxy design rather than contradictory to it: the guardrail thresholds encode cadence- and site-sensitive ramp, volatility, curvature, and mixing scales, so moving a frozen Wind (CN) calibration to a 5-minute site changes the operational meaning of the same screen before forecast skill is even compared. The supplement therefore turns to site-specific recalibration on Wind (US). With 5% target-site data, iTransformer reaches $R^2 = 0.8576$ and Ultra-LSNT + FG-MoE reaches 0.8458; with 10%,

the values are 0.8807 and 0.8612; with 50%, 0.8965 and 0.8920. Across the Ultra-LSNT recalibration runs, cut-region nonzero remains 0%. The second-site evidence supports recoverability after site-specific adaptation rather than calibration-free transfer.

Four limits remain. First, the main forecasting evidence still comes from one operational Wind (CN) site, and the Wind (US) supplement is a recalibration boundary rather than an independent positive validation site. Second, the corruption protocol, though operationally motivated, combines recorded SCADA irregularities with synthetic injection and a stress-envelope design, so it does not span the full space of real fault processes. Third, the operational readout is interface-specific and not AC-grade or market-grade. The main text reports only a fixed-interface point screen, and the rolling one-sided calibration diagnostic is left out of the main comparison because it remained unstable for ranking use [**?**]. Fourth, the deployment profile in Table **??** is generated in paper mode from a unified evidence bundle, but it remains pre-HIL rather than board-level evidence on target controller hardware [**?**]. Jump Gate calibration is also reported only on the clean-set controller search used for bundle selection, so a load-adaptive threshold policy remains future work. Within this scope, the paper provides one-site evidence of controller-side admissibility and burden reduction under a fixed interface for a site-calibrated guardrail.

7. Conclusion and outlook

On Wind (CN), FG-MoE improves controller-side admissibility for corrupted-SCADA forecasting. The core result is the within-stream ordering: relative to clip-only repair, FG-MoE removes above-rated violations while preserving usable skill, and the matched iTransformer reruns show that this effect is not confined to one carrier. Ultra-LSNT still matters as the audited sparse deployment path whose profiled software loop stays comfortably inside the 15-minute cadence, but the main methodological contribution lies in FG-MoE itself. Under the reported fixed interface, guarded streams also reduce slack burden, positive-error exposure, and real-time cost relative to their matched raw counterparts, even though the broader mapped ranking remains led by iTransformer.

The evidence remains one-site, pre-HIL, and interface-specific. Second-site gains require site-specific recalibration of cadence- and site-sensitive coefficients, and the Wind (US) results support recoverability rather than calibration-free transfer. The corruption protocol mixes logged irregularities with synthetic stress, while the operational readout remains a lower-bound screen

rather than AC-grade or market-grade proof. The reported deployment margin is software-loop headroom, not direct proof of execution inside a target PLC/RTU stack. Within this scope, the study shows that training-time feasibility guidance can improve controller-side admissibility and reduce burden under one fixed operational interface. The next steps are higher-fidelity AC-grade or market-grade valuation, followed by multi-site validation and then board-level or HIL confirmation on target controller hardware.

CRediT authorship contribution statement

Junyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Juntao Du**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Wind (CN): The Wind (CN) SCADA dataset used in this study is publicly available through the project repository (https://github.com/b1ue13e/Ultra-LSNT). The release includes the raw time series, the train/validation/test split indices, and the preprocessing scripts used to construct the learning task in Table **??**. A long-term archival mirror will be added upon acceptance.

Boundary-setting benchmark: The additional wind-site dataset used for second-site boundary checking is Wind (US) from the NREL Wind Toolkit [**?**]. It is used for boundary and recalibration diagnostics rather than as a second positive validation site for the main energy-focused claim.

Protocol definitions, supplementary tables, and run manifests are provided through the same repository and supplementary package. For the retained RTS-24 DC-OPF supplement, the released materials include the evaluation script, the hourly load profile, the bus

assignments, the wind-mapping coefficients, and the daily plus aggregate output tables under `results/supplementary_evidence/`.

Code: The implementation, training scripts, and evaluation protocols are available at https://github.com/b1ue13e/Ultra-LSNT under the MIT license.

Appendix A. Nomenclature and abbreviations

Item	Description
Abbreviations	
MoE	Mixture-of-Experts
SCADA	Supervisory Control and Data Acquisition
TSFM	Time-series foundation model
SOTA	State-of-the-art
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FG-MoE	Feasibility-Guided MoE (main-text term in this manuscript)
UQ	Uncertainty Quantification
CRPS	Continuous Ranked Probability Score
AC-OPF	AC Optimal Power Flow
DC-OPF	DC Optimal Power Flow
HIL	Hardware-in-the-Loop
Symbols	
L	input length (time steps)
H	prediction horizon (time steps)
C	number of input variables/covariates
$\mathbf{X} \in \mathbb{R}^{L \times C}$	input sequence
$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times 1}$	forecast output (wind power / target variable)
σ_{cfg}	configured perturbation ratio in robustness tests (dimensionless)
σ_{eff}	effective perturbation ratio, $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$
α	scaling factor used to map configured to effective noise severity ($\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, default $\alpha = 1.5$)
K	number of selected experts in Top-K routing
λ_r	load-balancing regularization coefficient for routing
τ	Jump Gate threshold for conditional skipping
R^2	coefficient of determination (scale-free)
RMSE/MAE	absolute errors in original physical units after inverse scaling
E_{inf}	estimated energy per sample inference (mJ)
B	batch size used in profiling (dimensionless)
N_{call}	number of forecasting calls per day

Item	Description
N_{turb}	number of turbines in a wind farm or fleet
c_f	fuel / purchase price coefficient in dispatch (CNY/kWh)
c_r	reserve price coefficient (CNY/kWh)
c_s	storage discharge cost coefficient (CNY/kWh)
c_{su}	start-up / shut-down cost coefficient (CNY)
$P_{\text{ch,max}}$	storage charging power limit (kW)
$P_{\text{dis,max}}$	storage discharging power limit (kW)
S_{max}	storage energy capacity limit (kWh)
R_{max}	thermal ramp-rate limit (kW per hour)
$\bar{P}_{\text{dev}}$	average device power during inference (W)
ρ	air density (kg/m^3), estimated from temperature/pressure
A	rotor swept area (m^2)
v	wind speed at hub height (m/s)
v_{t+h}^{proxy}	deployable horizon-aware wind proxy built from recent observed wind history
η_h	horizon-growth exponent in the proxy trend branch (dimensionless)
Δ_{max}	cap on extrapolated local ramp magnitude (m/s)
s_v	volatility scale activating the ramp-weight branch (m/s)
α_s, β_s	affine smoothing coefficients for the moving-average branch (dimensionless)
ω_p^{min}	persistence-floor coefficient before weight normalization (dimensionless)
$\omega_r^{\text{min}}, \omega_r^{\text{max}}$	lower / upper bounds on the ramp branch before weight normalization (dimensionless)
ξ_r	mixing coefficient between the trend and local-mean branches (dimensionless)
s_κ	curvature-damping scale in the proxy extrapolation (dimensionless)
$C_p(\lambda_{\text{tsr}}, \beta)$	power coefficient (Betz-limited)
λ_{tsr}	tip-speed ratio (dimensionless)
β	pitch angle (deg)
P_{rated}	rated-power upper bound (kW)
$P_{\text{env}}(v_{t+h}^{\text{proxy}})$	turbine envelope evaluated at the deployable wind proxy (kW)
$v_{\text{in}}, v_{\text{out}}$	cut-in / cut-out wind speeds (m/s)
λ_2	plausibility-penalty coefficient in heuristic FG-MoE (dimensionless)

Appendix B. Dispatch model details

This appendix gives the deterministic dispatch algebra used by the screening workflow in Section **??**. The equations below define the comparative screening model used throughout the manuscript. They are reported for reproducibility, not as AC-secure operational guidance.

With horizon $T = 24$ and $\Delta t = 1\,\mathrm{h}$, the objective is

$$\min\ C_{\text{dispatch}} = C_{\text{fuel}} + C_{\text{reserve}} + C_{\text{purchase}} + C_{\text{storage}} + C_{\text{startup}} + C_{\text{shutdown}}, \tag{B.1}$$

$$C_{\text{fuel}} = c_f \sum_{t=1}^{T} g_t \Delta t,$$

$$C_{\text{reserve}} = c_r \sum_{t=1}^{T} r_t \Delta t,$$

$$C_{\text{purchase}} = c_f \sum_{t=1}^{T} p_{\text{ch},t} \Delta t,$$

$$C_{\text{storage}} = c_s \sum_{t=1}^{T} p_{\text{dis},t} \Delta t,$$

$$C_{\text{startup}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_t - g_{t-1}, 0)}{R_{\max}},$$

$$C_{\text{shutdown}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_{t-1} - g_t, 0)}{R_{\max}}.$$

Subject to

$$g_t + \hat{w}_t + p_{\text{dis},t} - p_{\text{ch},t} - u_t = d_t, \tag{B.2}$$

$$0 \le u_t \le \hat{w}_t, \quad \forall t,$$

$$g_{\min} \le g_t \le g_{\max}, \tag{B.3}$$

$$0 \le r_t, \quad g_t + r_t \le g_{\max},$$

$$|g_t - g_{t-1}| \le R_{\max},$$

$$0 \le p_{\text{ch},t} \le P_{\text{ch,max}}, \tag{B.4}$$

$$0 \le p_{\text{dis},t} \le P_{\text{dis,max}},$$

$$s_t = s_{t-1} + \eta_{\text{ch}} p_{\text{ch},t} \Delta t - \eta_{\text{dis}}^{-1} p_{\text{dis},t} \Delta t, \tag{B.5}$$

$$0 \le s_t \le S_{\max}.$$

and the fixed-proportional reserve protocol

$$\sigma_{w,t} = 0.2 \cdot |\hat{w}_t|,\ t = 1, \ldots, T. \tag{B.6}$$

which is a deterministic comparison rule and not UQ-aware dynamic reserve optimization.

Table B.7: Reserve-ratio sensitivity under the fixed copper-plate interface on the 120-day Wind (CN) mapped screen. Negative deltas indicate lower burden than the matched raw stream.

ρ	Ultra clip-only ΔCost (%)	Ultra FG ΔCost (%)	Ultra FG ΔCurt. (%)	Ultra FG ΔReserve (%)	Ultra FG ΔSucc. (pp)	iTrans FG ΔCost (%)	iTrans FG ΔCurt. (%)
0.1	+0.03	-11.26	-77.96	-73.87	+31.67	-0.16	-1.21
0.2	+0.05	-18.54	-77.93	-73.87	+30.83	-0.27	-1.21
0.3	+0.07	-24.04	-77.63	-73.87	+25.83	-0.37	-1.21

Note: Rows are computed from the fixed-interface reserve-ratio sweep in `results/ae_dispatch_reserve_ratio_sensitivity_default120/`, with the same 80/20 Wind (CN) split manifest and the same mapped-and-clipped daily interface as the main copper-plate screen. The Ultra-LSNT clip-only control stays effectively on top of raw across the sweep. Ultra-LSNT + FG-MoE keeps a clear cost/curtailment/reserve advantage, while the matched iTransformer rerun preserves the same sign at much smaller magnitude.

Appendix B.1. Reserve-ratio sensitivity of the fixed interface

Table **??** repeats the same 120-day mapped-and-clipped copper-plate screen after replacing the baseline reserve rule $\sigma_{w,t} = 0.2|\hat{w}_t|$ with $\rho|\hat{w}_t|$, $\rho \in \{0.1, 0.2, 0.3\}$. All other ingredients are held fixed, including the seed-42 forecast archives and the Wind (CN) train-split upper bound $w_{\max}$. The table is therefore a stability check on the matched-stream operational ordering under a modest tightening or relaxation of reserve procurement.

Appendix C. Supplementary probabilistic and route-uncertainty diagnostics

This appendix gathers MoE-internal reserve heuristics and optional probabilistic heads used for interpretation. A rolling one-sided residual-calibration variant was also explored during revision, but under the fixed interface it did not produce a stable comparative ranking, so it is left outside the main evidence chain. The auxiliary conservative input is

$$\hat{w}_t^{\mathrm{lcb}} = \max(0, \hat{w}_t^{\mathrm{map}} - m_t), \tag{C.1}$$

where m_t is the rolling calibrated margin at hour t.

Router entropy and expert disagreement are defined as

$$H_t = -\sum_{i=1}^{N} p_{t,i} \log(p_{t,i} + \epsilon), \tag{C.2}$$

with $\epsilon = 10^{-8}$, and

$$D_t = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \left(\hat{y}_{t+h}^{(i)} - \bar{y}_{t+h} \right)^2, \tag{C.3}$$

$$\bar{y}_{t+h} = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \hat{y}^{(i)}_{t+h}. \tag{C.4}$$

These quantities can parameterize an auxiliary Ultra-specific reserve rule

$$\sigma_{w,t} = \min\Big(\sigma_{\max}, \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t\Big), \tag{C.5}$$

where σ_{base}, α_H, α_D, and $\sigma_{\max}$ are fixed coefficients. Because the rule depends on MoE-internal diagnostics, we treat it as an interpretive supplement rather than a main comparison layer. Optional probabilistic heads are

$$\mathcal{L}_{\text{pin}} = \frac{1}{|\mathcal{Q}|H} \sum_{q \in \mathcal{Q}} \sum_{h=1}^{H} \rho_q\Big(y_{t+h} - \hat{y}^{(q)}_{t+h}\Big), \tag{C.6}$$

$$\rho_q(e) = \max(qe, (q-1)e), \tag{C.7}$$

$$\mathcal{L}_{\text{nll}} = \frac{1}{H} \sum_{h=1}^{H} \left(\frac{(y_{t+h} - \mu_{t+h})^2}{2\sigma^2_{t+h}} + \log \sigma_{t+h} \right). \tag{C.8}$$

Appendix D. Implementation and environment details

Appendix D.1. Implementation details and computational environment

All experiments are implemented in Python (v3.10) using PyTorch (v2.0+) with CUDA (v11.8) acceleration. The core scientific stack includes NumPy (v1.24+) and Pandas (v2.0+) for data handling, Matplotlib (v3.7+) and Seaborn (v0.12+) for visualization, and LightGBM (v4.0+) with Scikit-learn (v1.3+) for tree-based baselines. These versions are listed so that both deep and non-deep baselines can be reproduced under consistent dependencies. The implementation is openly available at https://github.com/b1ue13e/Ultra-LSNT (MIT license), allowing inspection of model components such as sparse routing and conditional skipping and direct reproduction of the reported protocols.

Main training runs use a workstation equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel i9-13900K CPU, and 64 GB RAM for optimization and ablation sweeps. For batch-1 inference profiling, we additionally report results on commodity CPUs and on an ARMv8 (aarch64) single-core platform (Huawei Kunpeng 920). The Kunpeng run is included as a server-class ARM reference for cross-ISA comparison, not as a proxy for target edge hardware such as PLC-class controllers or embedded wind-farm gateways [**? ?**].

Implementation and training: We implement Ultra-LSNT in native PyTorch without third-party model dependencies. The sparse MoE core uses a custom Top-K routing layer ($K = 2$ in the main model), and the dynamic skip path uses a Jump Gate style conditional pass. To stabilize routing, the released configuration sets the auxiliary load-balancing coefficient to $\lambda = 0.01$ and keeps the router z-loss at 0.01, following standard MoE load-balancing and router-stabilization practice [**?**]; a controlled $\lambda \in \{0.01, 0.05\}$ robustness check is summarized in Table **??**. Unless otherwise stated, we train with $L = 96$, $H = 24$, chronological splits, train-fitted z-score normalization, AdamW $(\beta_1, \beta_2) = (0.9, 0.999)$, weight decay 0.01, cosine annealing with warm-up, initial learning rate 10^{-3}, and batch size 128; typical runs last 10–50 epochs depending on convergence.

Deployment profiling and reproducibility: The revised deployment evaluation uses a dedicated paper mode that exits immediately if the real Wind (CN) file or the released checkpoint is missing. Each run records the runtime environment, including PyTorch and ONNX Runtime versions, CPU identifier, thread count, warm-up iterations, and measured iterations, and writes structured logs into a unified deployment-evidence bundle. We profile fixed batch-1 inference for the deployment proxy candidates and for the full software loop reported in Table **??**. These timings stop at the software loop and therefore exclude controller integration layers such as fieldbus transfer, PLC scan scheduling, and board-level contention. Low-level runtime switches, manifests, and environment-specific optimization details are left to the bundled logs and supplementary material so the main manuscript can remain compact.

Appendix E. Robustness protocol details

Appendix E.1. Perturbation definitions and comparison setup

Additive Gaussian noise: We use additive Gaussian corruption as a controlled sensor-inaccuracy stress test, capturing short-lived measurement noise such as electromagnetic interference and turbulent sensing jitter in SCADA channels. Formally, for a clean input series x, we generate a corrupted series

$$x_{\text{noisy}} = x + \sigma_{\text{eff}} \, \text{std}(x) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 1), \tag{E.1}$$

where $\text{std}(x)$ is the standard deviation of the clean signal and σ_{eff} is the effective noise ratio. In our released noise generator, we use an industrial scaling factor $\alpha = 1.5$. To strengthen

empirical grounding, we additionally compute a channel-wise proxy noise ratio on Wind (CN) core channels (windspeed, winddirection, temperature, humidity, pressure, power):

$$r_c = \frac{\mathrm{std}(\Delta x_c)}{\sqrt{2}\,\mathrm{std}(x_c)}, \tag{E.2}$$

where $\Delta x_c = x_{t,c} - x_{t-1,c}$. On this dataset, the proxy statistics are median 0.045, P90 0.091, and max 0.121. Detailed calibration artifacts are archived in the supplementary evidence bundle.

Based on this anchor, we treat $\sigma_{\mathrm{eff}} = 0.15$ as an empirical upper-band neighborhood, while $\sigma_{\mathrm{eff}} \in \{0.30, 0.45, 0.60\}$ are explicit stress-envelope settings for robustness certification under abnormal SCADA conditions rather than nominal sensor-error claims. Throughout the manuscript, severity is reported by the *effective* ratio $\sigma_{\mathrm{eff}} = \alpha\sigma_{\mathrm{cfg}}$, so the injected noise has standard deviation $\sigma_{\mathrm{eff}} \cdot \mathrm{std}(x)$. In configuration files and scripts, the user-facing parameter is σ_{cfg}, so we report both σ_{cfg} and σ_{eff} when needed. Unless otherwise stated, the Gaussian sweep uses $\sigma_{\mathrm{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ (equivalently $\sigma_{\mathrm{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$).

Structured SCADA faults: Beyond Gaussian noise, real SCADA failures usually show temporal persistence and cross-channel dependence. To reflect that behavior, we synthesize three structured fault families on top of clean streams: flatlining (sensor-freeze windows generated by dwell-time processes such as geometric or semi-Markov sampling), spikes/heavy-tailed outliers (sparse impulsive corruption with amplitudes from Laplace or Student-t distributions), and correlated multi-channel dropouts (common-cause masking events such as DAQ resets that affect multiple variables jointly for contiguous windows). We evaluate these faults on Wind (CN) at severities 0.2 and 0.6; the severe slice is summarized in Table **??**, while full multi-severity rows are moved to supplementary material.

For reference against the deep models, we also run focused non-deep baselines (LightGBM and Persistence) over configured noise $\sigma_{\mathrm{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ (effective $\sigma_{\mathrm{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ with $\alpha = 1.5$), so severe-slice behavior can be viewed against both feature-engineered and naive-history references.

Drift and quantization noise: We also examine two perturbation families common in industrial sensing and edge pipelines. Drift noise approximates systematic bias that accumulates over time due to calibration drift, sensor aging, or persistent environmental interference, whereas quantization noise captures rounding and discretization errors introduced by low-precision telemetry, bandwidth-constrained transmission, or edge-side INT8/FP16 processing. To keep severity comparable, both are reported using the same effective ratio σ_{eff} as in the Gaussian protocol. Unless

otherwise stated, we sweep

$$\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}, \tag{E.3}$$

which corresponds to configured $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ when $\alpha = 1.5$.

Drift (random-walk bias): Given a clean channel $x_{1:L}$, we draw i.i.d. $\varepsilon_t \sim \mathcal{N}(0, 1)$ and define a persistent bias process

$$b_1 = 0. \tag{E.4}$$

$$b_t = b_{t-1} + \sigma_{\text{eff}} \cdot \text{std}(x) \cdot \varepsilon_t, \; t = 2, \ldots, L. \tag{E.5}$$

then corrupt the input as $x_t^{\text{drift}} = x_t + b_t$. The process is applied independently to each feature channel and each test sample. We reset $b_1 = 0$ per sample for paired reproducibility, and use the same random seed across methods at a given σ_{eff}; long contiguous fault persistence is covered separately by the structured-fault protocol.

Quantization (step-size rounding): We implement quantization as deterministic rounding to a channel-wise step size proportional to the clean-signal standard deviation:

$$\Delta = \sigma_{\text{eff}} \cdot \text{std}(x). \tag{E.6}$$

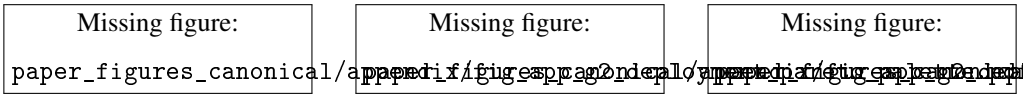
$$x_t^{\text{quant}} = \Delta \cdot \text{round}\left(\frac{x_t}{\Delta}\right). \tag{E.7}$$

where round$(\cdot)$ is round-to-nearest (half away from zero). When $\sigma_{\text{eff}} = 0$, no quantization is applied. This yields a reproducible, severity-controlled discretization without requiring dataset-specific bit-range tuning. The corresponding robustness outcomes are reported in Section **??** and interpreted in the Discussion with emphasis on reliability under realistic sensing constraints and energy-aware (Green AI) deployment.

Missingness and interpolation artifacts: Communication loss and sensor resets regularly create gaps in SCADA telemetry, after which simple forward-fill or linear interpolation is often applied. To emulate this setting, we sample a binary mask $m_{t,c} \sim \text{Bernoulli}(1 - p)$ over time steps t and channels c and define $\bar{x}_{t,c} = m_{t,c} x_{t,c} + (1 - m_{t,c})\text{NaN}$. We then apply an imputation operator Imp$(\cdot)$ to obtain the corrupted series

$$x^{\text{miss}} = \text{Imp}(\bar{x}; \texttt{ffill/linear}), \tag{E.8}$$

where `ffill` denotes forward filling and `linear` denotes piecewise-linear interpolation along the time axis. In the reported experiments, we use i.i.d. masking (no burst blocks) as a controlled



(a) Peak memory along the deployment optimization path.

(b) Severe-slice robustness on the same latency axis.

(c) Clean-slice accuracy on the same latency axis.

Figure G.5: Latency-aligned deployment summary. Panel (a) shows the controller-side optimization path from Ultra-Full FP32 to lighter carrier/export choices, while Panels (b)–(c) retain the same batch-1 latency axis so severe-slice robustness and clean-slice accuracy can be read against the same budget zone.

imputation stress test, sweep $p \in \{0.1, 0.3\}$, and repeat each configuration for three random seeds (0,1,2). For a fixed seed, the same missingness mask and imputation configuration are applied across models to preserve a paired comparison. Contiguous and correlated missingness is represented by the structured-fault dropout protocol. Results are reported in supplementary material.

Appendix F. Supplementary figure scope

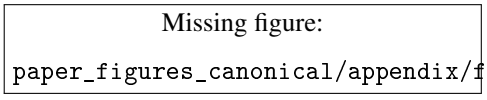
The supplementary figures start directly from deployment and operating-point diagnostics rather than repeating overview schematics or visual restatements of the matched-control robustness table.

Appendix G. Additional deployment and sensitivity figures

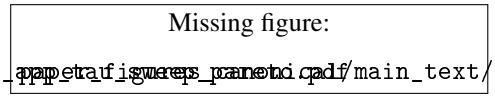
Figures **??**–**??** retain only the deployment and operating-point views that remain directly useful for choosing a deployment-consistent setting: one latency-aligned deployment summary and one merged sensitivity panel covering both the Jump Gate threshold sweep and the within-carrier λ_2 frontier.

Appendix H. Protocol scope and practical limits

Dispatch scope: Main-text dispatch results provide one fixed-interface operational readout from a fixed mapped-and-clipped forecast-to-grid rule. The RTS-24 layer is included as a coarse DC consistency screen. Mapping-sensitivity checks are reported separately, and the manuscript does not claim interface-independent operational superiority.



(a) Jump Gate threshold sweep.



(b) Within-carrier λ_2 frontier.

Figure G.6: Operating-point sensitivity after figure compression. Panel (a) links batch-1 CPU latency, predictive skill (R^2), and observed skip rate across Jump Gate thresholds; only the non-dominated operating points are connected, and $\tau = 0.7/0.9$ remain slower same-accuracy references relative to $\tau = 0.5$. Panel (b) shows the within-carrier λ_2 frontier under severe corruption with a symlog exceedance axis so true zero exceedance remains visible while preserving separation above 0.01%.

Hardware scope: Cross-ISA latency profiling is mixed-provenance and pre-HIL. Some rows are directly measured, others are retained/backfilled references, and the reported timings stop at the software loop without PLC scan scheduling, fieldbus transfer, or board-level contention. Board-level validation on target PLC/RTU hardware is still required.

Matched-control scope: This revision adds matched clean/fault-augmented controls for D-Linear, TimeMixer, iTransformer, and Ultra-LSNT (three seeds; four faults), but full coverage for long-tail supplementary baselines is left for future work.

Second-site scope: Wind (US) is used to mark the boundary between zero-shot failure and site-specific recovery. Zero-shot reuse fails for both retained models, while limited-data recalibration is reported to show recoverability rather than calibration-free transfer or model dominance.

Appendix I. Supplementary tables: ablations, interpretability, and significance

Appendix I.1. Evidence mapping for primary comparative numbers

Primary comparative numbers in this manuscript are taken from consolidated evidence tables rather than ad hoc extraction. Detailed file manifests remain in the public repository and supplement. The main text draws on those tables for robustness, safety, and shared-interface dispatch screening, while second-site recalibration and turbine-run rows are reported as supporting boundary evidence.

Appendix I.2. Within-carrier P2 operating-point table

Table **??** reports the P2 operating-point sweep for Ultra-LSNT under the SOA proxy. It is placed in the supplement because it characterizes an internal safety-accuracy trade-off within one

Table I.8: Supplementary Pareto operating points in (K, τ, λ_2) at $\sigma_{\text{eff}} = 0.60$ under protocol P2 (SOA proxy).

K	τ	λ_2	**Latency (ms)**	R^2@0.60	**Rated-power exceed. (SOA proxy, %)**
1	0.7	0.01	10.609	0.9301	19.1332
1	0.9	0.10	10.232	0.9257	18.0169
2	0.5	0.01	14.353	0.9294	19.0091
4	0.9	0.01	19.769	0.9299	18.9690
1	0.9	1.00	10.232	0.3831	0.0000
4	0.5	1.00	18.182	0.3913	0.0000

Note: Values are sampled from the retained P2 sensitivity sweep. The safety axis is rated-power exceedance under the protocol-P2 SOA proxy, so it is not numerically comparable to the protocol-P1 values in Table **??**.

Table I.9: Supplementary scale-error evidence at clean and $\sigma_{\text{eff}} = 0.60$ for directly logged Wind (CN) rows.

Model	**Mode**	**RMSE (clean)**	**MAE (clean)**	**RMSE@0.60**	**MAE@0.60**
DLinear	clean-trained	6065.98	3865.18	22190.19	15183.13
TimeMixer	clean-trained	9081.04	6789.63	12848.91	9674.21
iTransformer	clean-trained	7743.06	6029.31	9933.03	7852.08
Ultra-LSNT	clean-trained	9462.65	7360.92	10992.42	8705.76
DLinear	fault-augmented	6328.26	4181.08	20569.46	14105.63
TimeMixer	fault-augmented	8969.46	7067.89	10129.43	8034.08
iTransformer	fault-augmented	8779.48	6975.59	9593.71	7700.04
Ultra-LSNT	fault-augmented	9925.64	7858.85	10859.03	8675.17
Persistence	clean-trained	7313.94	4760.16	9485.55	6783.51
LightGBM	clean-trained	6880.91	4622.43	17665.50	13740.79
Chronos-Bolt-Small	zero-shot (512 ctx)	6216.19	3818.52	11443.18	8407.16
Moirai-2.0-R-Small	zero-shot (512 ctx)	5834.04	3460.46	11045.80	8176.62

Note: Matched-control rows for DLinear, TimeMixer, iTransformer, and Ultra-LSNT are included as Panel B companion evidence rather than as replacements for the Table **??** Panel A context panel. Chronos and Moirai use zero-shot extended-context inference (512-step input, same 24-step target window) and remain outside the controller-side deployability claim. Detailed source manifests are listed in the supplementary evidence bundle.

carrier rather than the main cross-model claim chain.

Appendix I.3. Scale-error supplementary table for robustness context

Table **??** provides supplementary RMSE/MAE evidence for the Wind (CN) context and matched-control rows used in this revision. The main table already reports RMSE in Panel A; Appendix G retains MAE and row coverage for completeness, not to replace the primary comparison.

Table I.10: Site-calibrated Wind (CN) proxy parameter sensitivity rerun for FG-MoE under the shared high-risk interface (seed 42).

Case	s_v	$\Delta_{\max}$	$\omega_r^{\min}$	$\omega_r^{\max}$	**Clean R^2**	R^2@0.60	**Severe over-rated (%)**	**Slack (MWh)**	**RT cost (M CNY)**
Baseline fixed	0.850	1.250	0.180	0.720	0.8725	0.8513	0.0062	582.5	30.47
s_v low	0.595	1.250	0.180	0.720	0.8723	0.8511	0.0081	567.2	30.44
s_v high	1.105	1.250	0.180	0.720	0.8724	0.8511	0.0081	567.3	30.44
$\Delta_{\max}$ low	0.850	0.875	0.180	0.720	0.8725	0.8511	0.0106	582.5	30.47
$\Delta_{\max}$ high	0.850	1.625	0.180	0.720	0.8725	0.8511	0.0106	582.5	30.47
ω_r low band	0.850	1.250	0.126	0.504	0.8725	0.8522	0.0129	582.5	30.47
ω_r high band	0.850	1.250	0.234	0.936	0.8723	0.8509	0.0196	567.7	30.44

Note: All rows use the full 120-day Wind (CN) shared high-risk point readout and differ in only one coefficient group at a time. Across these reruns, clean R^2 stays within 0.8723–0.8725, severe-slice R^2 within 0.8509–0.8522, and the high-risk burden endpoints stay in the same narrow range. The corresponding clip-only control remains much weaker on the clean slice (R^2 = 0.4335–0.4337), so the rerun is better read as evidence of ordering stability than as exact numerical reproduction of the canonical baseline row.

Missing figure:

cross_domain_run_artifacts_20260330_184430/results/cross_domain/domain_adaptation_curve.png

Figure I.7: Wind (US) second-site recalibration curves using limited target-site data after Wind (CN) pretraining. The figure is included to show recoverability after site-specific recalibration rather than calibration-free transfer.

Appendix I.4. Site-calibrated proxy parameter sensitivity table

Table **??** reports the site-calibrated Wind (CN) proxy sensitivity rerun used to check whether the FG-MoE guardrail effect is stable under moderate perturbations of the deployable proxy parameters. Each row reuses the same seed-42 training/evaluation chain and the same shared high-risk dispatch interface as the main results.

Appendix I.5. Second-site recalibration diagnostic

Wind (US) is used to mark the boundary between zero-shot failure and site-specific recovery. We separate two cases. First, the zero-shot reruns fail for both retained models and do not support any transfer claim. Second, once a small amount of target-site data is introduced for recalibration, both retained models recover to positive skill, although iTransformer remains slightly stronger on pure R^2.

Table I.11: Second-site recalibration summary on Wind (US) after Wind (CN) pretraining.

Panel A

Zero-shot boundary statement

Case	Ultra-LSNT + FG-MoE	iTransformer
Wind (US) zero-shot reuse	failure; no transfer claim	failure; no transfer claim

Panel B

Few-shot site recalibration on Wind (US)

Target-site data	Ultra-LSNT + FG-MoE R^2	Ultra cut-region nonzero (%)	iTransformer R^2
1%	0.5363	0.0	0.6829
5%	0.8458	0.0	0.8576
10%	0.8612	0.0	0.8807
50%	0.8920	0.0	0.8965
100%	0.8893	0.0	0.8959

Note: Panel A records the zero-shot boundary check and does not report a positive transfer result. Panel B reports the site-specific recalibration summary traced to `summary_domain_adaptation.csv`. The table is kept to show recoverability after target-site recalibration. iTransformer remains slightly stronger on pure R^2, while Ultra-LSNT keeps zero cut-region nonzero throughout the recalibration runs.

Appendix I.6. Full structured SCADA fault robustness table

Complete multi-severity and multi-metric structured-fault results for Wind (CN) are reported in the supplementary package.

Appendix I.7. Observed turbine fault-bank diagnostic table

To complement the main Wind (CN) stress tests, we report additional turbine-side diagnostics on the prepared `wind_turbine_primary` operational run file. The observed-fault bank contains 343 recorded events: 247 wind-speed flatlines, 50 wind-speed spikes, 40 power spikes, and 6 power flatlines; flatline episodes average 6.42 samples and reach a maximum length of 49. These rows remain secondary because the turbine file uses a different feature basis from Wind (CN), so checkpoints must be warm-initialized with shape-aware partial loading before turbine-side retraining. Even under that condition, the separate guardrail replay preserves the clip-only versus FG-MoE ordering on the turbine file: FG-MoE remains the only variant with positive clean/severe skill (0.7486/0.7107), whereas clip-only drops below −0.90 on both slices. A separate 10-seed safety repeat on the same file preserves positive FG-MoE mean clean/severe

Table I.12: Secondary diagnostic evidence on an independent turbine run file under the matched full-fault protocol (3 seeds).

Model	**Clean R^2**	**Drift R^2**	**Gauss. R^2**	**Fault-aug clean R^2**	**Fault-aug drift R^2**	**Fault-aug Gauss. R^2**
DLinear	0.7103	0.3986	−1.4697	0.6910	0.5277	−1.2200
TimeMixer	0.7387	0.3664	0.4998	0.7253	0.6811	0.6721
iTransformer	0.7336	0.5650	0.6665	0.7068	0.6997	0.6885
Ultra-LSNT	0.6934	0.4204	0.6339	0.6704	0.6591	0.6565

Note: Rows are taken from the turbine full-run table under the same $(L, H) = (96, 24)$ protocol as the main manuscript and are reported as secondary observed-fault diagnostics rather than as an extension of the Wind (CN) headline comparison. Fault augmentation improves drift and Gaussian robustness for Ultra-LSNT on this file, but TimeMixer and iTransformer retain stronger absolute R^2 under the same matched closure. The complementary turbine-side guardrail replay is stored in `fgmoe_clip_only_control_turbine_fullrun.csv` and `table_safety_coupling_turbine_fullrun.csv`; the repeated-seed safety summary is stored separately in the 10-seed turbine bundle.

R^2 of 0.7038/0.6801 with mean P95 latency of 14.92/15.12 ms, while raw and clip-only variants remain negative on both slices. The matched full-fault comparison appears in Table **??**.

Appendix I.8. Diagnostic long-tail baseline matrix (diagnostic-only)

For transparency, we retain broad baseline rows, including traditional methods, hybrid methods, and constrained-budget metaheuristics, in the supplementary evidence bundle. These rows are diagnostic only and are not used in the matched-control main comparisons.

Appendix I.9. Ablation results

All ablations are conducted under a fixed training budget and identical evaluation protocol; we interpret them primarily through relative ranking and directional sensitivity, rather than expecting the absolute R^2 to exactly match the full benchmark runs.

Supplementary ablation tables report kernel-size sensitivity from the retained ablation table. On Wind (CN), $k = 3$ and $k = 7$ are close, while $k = 15$ is weaker. This supports using compact receptive fields for turbulence-dominant SCADA dynamics. Router-regularization robustness is reported in Table **??** using the two retained λ checkpoints (0.01 and 0.05).

Routing diagnostics for operational interpretation are presented in the main text (Section **??**, Figure **??**).

Table I.13: Diebold–Mariano test results on Wind (CN) using squared-error loss; DM< 0 favors Ultra-LSNT.

Comparison	DM statistic	p-value
Ultra-LSNT vs PatchTST	-5.90	1.82×10^{-9}
Ultra-LSNT vs TimeMixer	3.45	2.83×10^{-4}
Ultra-LSNT vs iTransformer	5.12	3.07×10^{-7}

Appendix I.10. Statistical significance

We assess whether performance gaps are statistically significant using the Diebold–Mariano (DM) test on squared prediction errors [**?**]. Let $d_t = \ell_t(\text{Ultra-LSNT}) - \ell_t(\text{Baseline})$ denote the per-time step loss differential computed on the same test split; hence DM< 0 indicates Ultra-LSNT has smaller average loss, while DM> 0 favors the baseline. This significance check complements the descriptive context in Table **??**. Table **??** summarizes the paired Wind (CN) DM contrasts retained in this revision. Across the three retained contrasts, we apply a Bonferroni-adjusted family-wise threshold of $\alpha = 0.05/3 = 0.0167$; all three reported p values remain below it. The DM check is best read as direction-stability evidence under fixed splits, not as a replacement for the protocol-bound robustness and dispatch screening claims in the main results.

The retained DM contrasts indicate statistically significant paired loss differentials across representative comparisons. They support a narrower point than a ranking-style claim: clean-slice paired error ordering can differ from the controller-side ordering that appears once corrupted-input robustness and deployability constraints are imposed. Scripts and the loss-differential series for the full model set are provided in the supplementary material. We therefore treat the DM results as supporting evidence and keep robustness stress tests plus fixed-interface operational screening as the more informative basis for the main claims.

Generative AI and AI-assisted technologies statement

During preparation of this manuscript, the authors used *ChatGPT* (OpenAI), under direct author supervision, for language polishing, figure-pipeline scripting support, and visual layout drafting. Scientific interpretation, data selection, numerical reporting, figure content, and final manuscript decisions were all performed and verified by the authors, who take full responsibility

for the published content. No generative AI tools were used to generate or alter the experimental data itself.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] J. Wang, A. Botterud, R. Bessa, H. Keko, L. Carvalho, D. Issicaba, J. Sumaili, V. Miranda, Wind power forecasting uncertainty and unit commitment, Applied Energy 88 (11) (2011) 4014–4023.

[2] N. Houben, A. Cosic, M. Stadler, M. Mansoor, M. Zellinger, H. Auer, A. Ajanovic, R. Haas, Optimal dispatch of a multi-energy system microgrid under uncertainty: A renewable energy community in austria, Applied Energy 337 (2023) 120913. doi:10.1016/j.apenergy.2023.120913.

[3] J. Yan, X. Li, H. Wang, F. Si, W. Qiao, Y. Liu, The economic value of wind power forecasting: a data-driven method and its application in various scenarios, Energy 340 (2025) 139151. doi:10.1016/j.energy.2025.139151.

[4] H. Khaloie, A. Abdollahi, M. Shafie-khah, A. Anvari-Moghaddam, S. Nojavan, P. Siano, J. P. Catalão, Coordinated wind-thermal-energy storage offering strategy in energy and spinning reserve markets using a multi-stage model, Applied Energy 259 (2020) 114168.

[5] C. Li, A. Kies, K. Zhou, M. Schlott, O. E. Sayed, M. Bilousova, H. Stöcker, Optimal power flow in a highly renewable power system based on attention neural networks, Applied Energy 359 (2024) 122779. doi:10.1016/j.apenergy.2024.122779.

[6] J. Tautz-Weinert, S. J. Watson, Using SCADA data for wind turbine condition monitoring: A review, IET Renewable Power Generation 11 (4) (2017) 382–394. doi:10.1049/iet-rpg.2016.0248.

[7] K. Leahy, C. Gallagher, P. O'Donovan, D. T. J. O'Sullivan, Issues with data quality for wind turbine condition monitoring and reliability analyses, Energies 12 (2) (2019) 201. doi:10.3390/en12020201.

[8] J. Maldonado-Correa, S. Martín-Martínez, E. Artigao, E. Gómez-Lázaro, Using SCADA data for wind turbine condition monitoring: A systematic literature review, Energies 13 (12) (2020) 3132. doi:10.3390/en13123132.

[9] L. Sheng, C. Li, M. Gao, X. Xi, D. Zhou, A review of SCADA-based condition monitoring for wind turbines via artificial neural networks, Neurocomputing 633 (2025) 129830. doi:10.1016/j.neucom.2025.129830.

[10] I. Vervlimmeren, X. Chesterman, T. Verstraeten, A. Nowé, J. Helsen, Scalable scada-driven failure prediction for offshore wind turbines using autoencoder-based nbm and fleet-median filtering, Wind Energy Science 10 (2025) 2615–2637. URL https://wes.copernicus.org/articles/10/2615/2025/

[11] P. Martín-Calzada, P. Martín Sánchez, F. J. Rodríguez-Sánchez, C. Santos-Pérez, J. Ballesteros, Optimized sensor data preprocessing using parameter-transfer learning for wind turbine power curve modeling, Sensors 25 (17) (2025) 5329. doi:10.3390/s25175329.

[12] A. Verdone, M. Panella, E. De Santis, A. Rizzi, A review of solar and wind energy forecasting: From single-site to multi-site paradigm, Applied Energy 392 (2025) 126016. doi:10.1016/j.apenergy.2025.126016.

[13] Y. Zhang, K. Liu, J. Qian, Wind power prediction based on variational mode decomposition and artificial intelligence algorithms: A review, Energy Conversion and Management 227 (2021) 113615. doi:10.1016/j.enconman.2020.113615.

[14] W. Duan, Z. Zhang, A. Zhong, Z. Tang, A robust wind power forecasting framework for non-stationary signals via decomposition and metaheuristic optimization, Energies 18 (24) (2025) 6515. doi:10.3390/en18246515.

[15] Y. Zhu, Y. Liu, N. Wang, Z. Zhang, Y. Li, Real-time error compensation transfer learning with echo state networks for enhanced wind power prediction, Applied Energy 379 (2025) 124893. doi:10.1016/j.apenergy.2024.124893.

[16] Z. Ma, G. Mei, A hybrid attention-based deep learning approach for wind power prediction, Applied Energy 323 (2022) 119608. doi:10.1016/j.apenergy.2022.119608.

[17] M. Gao, S. Zhou, W. Gu, Z. Wu, H. Liu, A. Zhou, X. Wang, Mmgpt4lf: Leveraging an optimized pre-trained gpt-2 model with multi-modal cross-attention for load forecasting, Applied Energy 392 (2025) 125965. doi:10.1016/j.apenergy.2025.125965.

[18] T. Wu, Q. Ling, Stellm: Spatio-temporal enhanced pre-trained large language model for wind speed forecasting, Applied Energy 375 (2024) 124034. doi:10.1016/j.apenergy.2024.124034.

[19] H. Liu, C. Chen, Data processing strategies in wind energy forecasting models and applications: A comprehensive review, Applied Energy 249 (2019) 392–408. doi:10.1016/j.apenergy.2019.04.188.

[20] S. Wang, B. Li, G. Li, B. Yao, J. Wu, Short-term wind power prediction based on multidimensional data cleaning and feature reconfiguration, Applied Energy 292 (2021) 116851. doi:10.1016/j.apenergy.2021.116851.

[21] W. Yang, R. Court, J. Jiang, Wind turbine condition monitoring by the approach of SCADA data analysis, Renewable Energy 53 (2013) 365–376. doi:10.1016/j.renene.2012.11.030.

[22] P. Bangalore, M. Patriksson, Analysis of SCADA data for early fault detection, with application to the maintenance management of wind turbines, Renewable Energy 115 (2018) 521–532. doi:10.1016/j.renene.2017.08.073.

[23] S. Letzgus, Change-point detection in wind turbine SCADA data for robust condition monitoring with normal behaviour models, Wind Energy Science 5 (2020) 1375–1397. doi:10.5194/wes-5-1375-2020.

[24] N. Renström, P. Bangalore, E. Highcock, System-wide anomaly detection in wind turbines using deep autoencoders, Renewable Energy 157 (2020) 647–659. doi:10.1016/j.renene.2020.04.148.

[25] J. Tawn, E. Browell, I. Dinwoodie, Missing data in wind farm time series: Properties and effect on forecasts, Electric Power Systems Research 189 (2020) 106640. doi:10.1016/j.epsr.2020.106640.

[26] H. Wen, R. McCrea, E. J. Browell, Wind energy forecasting with missing values within a fully conditional specification framework, International Journal of Forecasting 40 (4) (2024) 1361–1382. doi:10.1016/j.ijforecast.2022.12.006.

[27] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, Proceedings of the IEEE 107 (8) (2019) 1738–1762. doi:10.1109/JPROC.2019.2918951.

[28] H. B. Gooi, Z. Wang, Y. Tang, Edge intelligence for smart grid: A survey of applications, techniques and challenges, CSEE Journal of Power and Energy Systems 9 (5) (2023) 1623–1640. doi:10.17775/CSEEJPES.2022.02210.

[29] A. Mwangi, R. Sahay, E. Fumagalli, M. Gryning, M. Gibescu, Towards a software-defined industrial IoT-edge network for next-generation offshore wind farms: State of the art, resilience, and self-x network and service, Energies 17 (12) (2024) 2897. doi:10.3390/en17122897.

[30] D. Astolfi, F. Castellani, A. Garinei, L. Terzi, Data mining techniques for performance analysis of onshore wind farms, Applied Energy 148 (2015) 220–233. doi:10.1016/j.apenergy.2015.03.075.

[31] Y. Liu, J. Wang, L. Liu, Physics-informed reinforcement learning for probabilistic wind power forecasting under extreme events, Applied Energy 376 (2024) 124068.

[32] W. Liao, J. Fang, L. Ye, B. Bak-Jensen, Z. Yang, F. Porté-Agel, Can we trust explainable artificial intelligence in wind power forecasting?, Applied Energy 376 (2024) 124273. doi:10.1016/j.apenergy.2024.124273.

[33] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, G. E. Hinton, Adaptive mixtures of local experts, Neural Computation 3 (1) (1991) 79–87. doi:10.1162/neco.1991.3.1.79.

[34] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models, Journal of Machine Learning Research 23 (2022) 1–39.

[35] D. Davis, M. J. Brear, Impact of short-term wind forecast accuracy on the performance of decarbonising wholesale electricity markets, Energy Economics 130 (2024) 107304. doi:10.1016/j.eneco.2024.107304.

[36] H. Quan, D. Srinivasan, A. M. Khambadkone, A. Khosravi, A computational framework for uncertainty integration in stochastic unit commitment with intermittent renewable energy sources, Applied Energy 152 (2015) 71–82.

[37] Y. Guo, X. Han, X. Zhou, G. Hug, Incorporate day-ahead robustness and real-time incentives for electricity market design, Applied Energy 332 (2023) 120484. doi:10.1016/j.apenergy.2022.120484.

[38] A. Srinivasan, R. Wu, P. Heer, G. Sansavini, Impact of forecast uncertainty and electricity markets on the flexibility provision and economic performance of highly-decarbonized multi-energy systems, Applied Energy 338 (2023) 120825. doi:10.1016/j.apenergy.2023.120825.

[39] M. Lechl, H. de Meer, T. Fürmann, A stochastic flexibility calculus for uncertainty-aware energy flexibility management, Applied Energy 379 (2025) 124907. doi:10.1016/j.apenergy.2024.124907.

[40] G. Terrén-Serrano, M. Ludkovski, Extreme day-ahead renewables scenario selection in power grid operations, Applied Energy 391 (2025) 125747. doi:10.1016/j.apenergy.2025.125747.

[41] W. Huo, Y. Zhang, H. Zhao, F. Lin, J. Wang, Price signal forecasting for day-ahead offering strategy of price-maker renewable producers considering different risk preferences, Applied Energy 401 (2025) 126819. doi:10.1016/j.apenergy.2025.126819.

[42] J. Liu, L. Wang, T. Jiao, W. Yang, R. Xu, K. Wu, H. Ha, Multi-period voltage stability-constrained optimal power flow with uncertainties, Applied Energy 369 (2024) 123522. doi:10.1016/j.apenergy.2024.123522.

[43] D. Larrahondo, R. Moreno, H. R. Chamorro, F. Gonzalez-Longatt, Comparative performance of multi-period acopf and multi-period dcopf under high integration of wind power, Energies 14 (15) (2021) 4540. doi:10.3390/en14154540.

[44] H. Zhang, G. Qin, X. Chen, L. Lu, Z. Zhang, J. Song, A day-ahead wind power dynamic explainable prediction method based on shap analysis and mixture of experts, Energies 19 (1) (2025) 124. doi:10.3390/en19010124.

[45] X. Liu, J. Liu, G. Woo, T. Aksu, Y. Liang, R. Zimmermann, C. Liu, S. Savarese, C. Xiong, D. Sahoo, Moirai-MoE: Empowering time series foundation models with sparse mixture of experts, in: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Vol. 267 of Proceedings of Machine Learning Research, 2025, pp. 38940–38962.
URL https://proceedings.mlr.press/v267/liu25an.html

[46] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, M. Jin, Time-moe: Billion-scale time series foundation models with mixture of experts, in: International Conference on Learning Representations (ICLR), 2025, pp. 1–33.
URL https://openreview.net/forum?id=e1wDDFmlVu

[47] T. Burton, N. Jenkins, D. Sharpe, E. Bossanyi, Wind Energy Handbook, 2nd Edition, Wiley, 2011.

[48] International Electrotechnical Commission, IEC 61400-12-1:2017 wind energy generation systems – part 12-1: Power performance measurements of electricity producing wind turbines, International Standard, edition 2.0 (2017).
URL https://webstore.iec.ch/en/publication/26603

[49] S. M. Parsa, Physics-informed machine learning meets renewable energy systems: A review of advances, challenges, guidelines, and future outlooks, Applied Energy 402 (2025) 126925.

[50] Y. Wang, F. Zhang, H. Kou, R. Zou, Q. Hu, J. Wang, D. Srinivasan, A review of predictive uncertainty modeling techniques and evaluation metrics in probabilistic wind speed and wind power forecasting, Applied Energy 396 (2025) 126234.
doi:10.1016/j.apenergy.2025.126234.

[51] S. Liang, S. Jin, Y. Chen, A review of edge computing technology and its applications in power systems, Energies 17 (13) (2024) 3230. doi:10.3390/en17133230.

[52] Y. Chen, Z. Yu, Y. Jin, C. Mwase, X. Hu, L. D. Xu, Z. Zou, L. Zheng, Self-aware collaborative edge inference with embedded devices for IIoT, Future Generation Computer Systems 163 (2025) 107535. doi:10.1016/j.future.2024.107535.

[53] L. A. Pazmiño Ortiz, I. F. Maldonado Soliz, V. K. Guevara Balarezo, Advancing TinyML in IoT: A holistic system-level perspective for resource-constrained AI, Future Internet 17 (6) (2025) 257. doi:10.3390/fi17060257.

[54] T. Li, J. Yang, A. Ioannou, Wind forecasting-based model predictive control of generator, pitch, and yaw for output stabilisation – a 15-megawatt offshore, Energy Conversion and Management 302 (2024) 118155. doi:10.1016/j.enconman.2024.118155.

[55] Y. Guo, H. Gao, Q. Wu, J. Østergaard, D. Yu, M. Shahidehpour, Distributed coordinated active and reactive power control of wind farms based on model predictive control, International Journal of Electrical Power & Energy Systems 104 (2019) 78–88. doi:10.1016/j.ijepes.2018.06.043.

[56] K. Li, Y. Mu, F. Yang, H. Wang, Y. Yan, C. Zhang, A novel short-term multi-energy load forecasting method for integrated energy system based on feature separation-fusion technology and improved cnn, Applied Energy 351 (2023) 121823. doi:10.1016/j.apenergy.2023.121823.

[57] K. Dragomiretskiy, D. Zosso, Variational mode decomposition, IEEE Transactions on Signal Processing 62 (3) (2014) 531–544. doi:10.1109/TSP.2013.2288675.

[58] Z. Wu, N. E. Huang, Ensemble empirical mode decomposition: A noise-assisted data analysis method, Advances in Adaptive Data Analysis 1 (1) (2009) 1–41. doi:10.1142/S1793536909000047.

[59] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.
URL https://doi.org/10.1145/3381831

[60] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational

Linguistics, 2019, pp. 3645–3650. doi:10.18653/v1/P19-1355.
URL https://aclanthology.org/P19-1355/

[61] A. Botterud, Z. Zhou, Use of wind power forecasting in operational decisions, Tech. rep., Argonne National Laboratory, technical report (2011).

[62] National Energy Administration of China, DL/T 5002-2021 Design code for dispatch automation of regional power grids, DL/T 5002-2021, industry standard issued by the National Energy Administration of China; implemented on 2021-10-26 (in Chinese). (2021).

[63] J. Ren, W. Guo, Z. Liu, M. Wu, X. Du, A review of scada systems in power grids: Security, architecture, and perspectives, IEEE Internet of Things Journal 8 (16) (2021) 12741–12759. doi:10.1109/JIOT.2020.3044893.

[64] C. Draxl, A. Clifton, B.-M. Hodge, J. McCaa, The wind integration national dataset (wind) toolkit, Applied Energy 151 (2015) 355–366. doi:10.1016/j.apenergy.2015.03.121.

[65] F. X. Diebold, R. S. Mariano, Comparing predictive accuracy, Journal of Business & Economic Statistics 13 (3) (1995) 253–263. doi:10.1080/07350015.1995.10524599.